Politecnico di Torino

Master of Science in Computer Engineering

Department of Control and Computer Engineering (DAUIN)

Database and Data Mining Group (DBDMG)

Large Language Models for Software Engineering

Volume II: Hands-on Laboratory Practicum

PyTorch foundations, Hugging Face architectures, PEFT, RAG, and SE agents

Complete laboratory implementation guide with full solutions

Step-by-step code walkthroughs, mathematical autograd derivations, and verification

Course Instructors:
Prof. Flavio Giobergia
Prof. Riccardo Coppola

Laboratory Curators:
Claudio Savelli
Tommaso Fulcini

Politecnico di Torino

Academic Year:
2026 / 2027

Target Program:
M.Sc. Computer Engineering

Course Credits:
6 CFU

Typeset with LaTeX, `tcolorbox`, and PGF/TikZ
Torino, Italy — A.Y. 2026/2027

Contents

I Foundations of deep learning and transformers **1**

1 Laboratory 01: Deep learning foundations with PyTorch **2**
1.1 Laboratory overview and macro perspective in LLM systems 2
1.2 PyTorch tensors, memory storage, and precision benchmarking 4
1.3 Computational graphs, autograd, and sensitivity mechanics 6
1.4 Building neural models, datasets, and the canonical training loop 9
1.5 Computer vision pipeline and data augmentation (CIFAR-10) 13
1.6 Multi-layer perceptron for classification (`SimpleNN`) 15
1.7 Transfer learning and parameter freezing with ResNet-18 17
1.8 Exercise 3 in-depth: analytical derivation and autograd DAG trace 20
1.9 Laboratory summary and engineering checklist 21

2 Laboratory 02: Hugging Face ecosystem, encoder-decoder models, and attention mechanisms **24**
2.1 Laboratory overview and macro perspective in LLM systems 24
2.2 Tokenization pipelines, subwords, and batch padding mechanics 26
2.3 Anatomy of the T5 architecture and embedding geometry 30
2.4 Transformer encoder and decoder blocks in T5 32
2.5 Autoregressive token generation and decoding dynamics 34
2.6 Cross-attention weight extraction and interpretability 36
2.7 Scaled dot-product attention from scratch: Mathematical foundations 38
2.8 Synthetic time series modeling with periodic jumps 39
2.9 Relative vs absolute positional encodings: Theory and empirical proof 44
2.10 Instructor insights, exam takeaways, and engineering checklist 47

II Large language models for software engineering **50**

Part I

Foundations of deep learning and transformers

CHAPTER 1

Laboratory 01: Deep learning foundations with PyTorch

1.1 Laboratory overview and macro perspective in LLM systems

Before building modern foundation models with tens of billions of parameters, an AI software engineer must master the computational primitives that underpin all deep learning. Every Large Language Model (LLM)—from decoder-only transformers like LLaMA and GPT-4 to encoder-decoder models like T5—is executed on top of tensor arithmetic, automatic differentiation engines, and hardware-accelerated memory hierarchies.

1.1.1 Where this laboratory fits in the LLM lifecycle

To understand why we begin with PyTorch tensors, autograd, and transfer learning, consider the macroscopic lifecycle of an LLM shown in Figure 1.1.

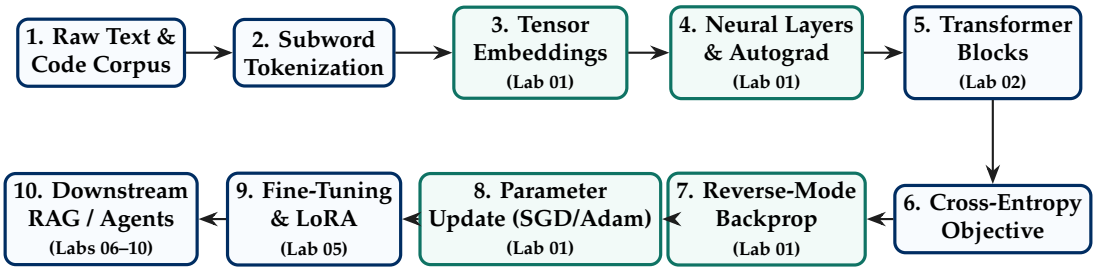


Figure 1.1: The end-to-end lifecycle of Large Language Models. Laboratory 01 establishes the atomic computational substrate: multidimensional tensors (Step 3), modular neural layers and forward graphs (Step 4), reverse-mode autograd (Step 7), and iterative optimization loops (Step 8).

Regardless of whether a model processes computer vision pixels ($32 \times 32 \times 3$) or textual tokens ($B \times T \times d_{\text{model}}$), the underlying execution pattern is strictly invariant across machine learning domains:

1. Input data is projected into multidimensional floating-point **tensors**.
2. A parameterized computational model (`nn.Module`) applies affine transformations interspersed with non-linear activations.
3. A scalar **loss function** evaluates prediction discrepancies against target ground truth.
4. An **automatic differentiation engine** traverses the Directed Acyclic Graph (DAG) in reverse, computing exact partial derivatives $\nabla_\theta \mathcal{L}$.
5. An **optimizer** updates parameters along the negative gradient surface.

1.1.2 Curriculum structure and pedagogical roadmap

As emphasized during the introductory laboratory lecture, the laboratory practicum adopts a progressive, modular structure:

- **Laboratory 01 (Foundational substrate)**: A self-contained, single-part introductory practicum establishing tensor manipulation, autograd mechanics, loss formulations, training loops, and transfer learning fundamentals.
- **Laboratories 02–10 (Dual-part advanced practicums)**: Starting from Laboratory 02, every subsequent session is partitioned into two complementary modules:
  1. *Architectural implementation from scratch* (e.g., implementing multi-head self-attention, positional encodings, subword tokenization, or LoRA adapters).
  2. *High-level industrial frameworks* (e.g., Hugging Face libraries such as `transformers`, `peft`, `trl`, and `accelerate`).

> 🎓 **Exam insights: Exam syllabus scope: NLP vs computer vision in LLM4SE**
>
> During the live laboratory session, the instructors explicitly clarified the examination boundaries: *students will not be evaluated on computer vision details, image data augmentation primitives (`RandomCrop`, `RandomHorizontalFlip`), or convolutional feature maps in the final exam.*
> Computer vision datasets (specifically CIFAR-10) are utilized exclusively in this first laboratory as an accessible, visually intuitive sandbox for high-dimensional tensors ($3 \times 32 \times 32$), batch slicing, multi-channel normalization, and pretrained transfer learning before students transition to discrete subword tokens, vocabulary embeddings, and causal attention masks.

1.1.3 Laboratory learning outcomes

Upon completing this laboratory, you will be able to:

- Manipulate PyTorch tensors, analyze underlying memory layouts (strides and contiguity), and evaluate hardware precision trade-offs (FP64 vs. FP32 vs. BF16).
- Trace and visualize Directed Acyclic Computational Graphs (DAGs) using reverse-mode automatic differentiation.
- Subclass `torch.nn.Module`, implement custom parameter layers, and configure optimizers and loss functions.
- Author robust, multi-epoch training loops from scratch with metric tracking and loss visualization.
- Compute per-channel dataset statistics (mean and standard deviation) and build data transformation and augmentation pipelines with `torchvision.transforms`.
- Implement multi-layer perceptron classifiers (`SimpleNN`) and construct evaluation loops with `model.eval()` and `torch.no_grad()`.
- Apply transfer learning with pretrained deep residual networks (ResNet-18), freeze convolutional feature extractors, and adapt classification heads for new downstream domains.

1.2 PyTorch tensors, memory storage, and precision benchmarking

In deep learning, a **tensor** is a generalized geometric object comprising a multidimensional array of numbers characterized by a specific data type (`dtype`) and memory device (`device`).

1.2.1 Importing the environment and OpenMP configuration

To ensure robust multi-threaded tensor execution across Apple Silicon and Linux environments, we initialize the required libraries and set environment flags:

```
# Import necessary libraries
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
import matplotlib.pyplot as plt

import torchvision
import torchvision.transforms as transforms
```

Listing 1.1: Initial library imports and environment configuration (Cell 2).

1.2.2 Basic tensor algebra

A PyTorch tensor acts as a multidimensional view over a contiguous chunk of flat physical memory:

```
# Create a tensor
a = torch.tensor([1.0, 2.0, 3.0])

# Perform basic tensor operations
b = torch.tensor([4.0, 5.0, 6.0])
c = a + b  # Element-wise addition
print(f"Tensor c: {c}")

# Check tensor's shape and type
print(f"Shape of tensor c: {c.shape}")
print(f"Data type of tensor c: {c.dtype}")
```

Listing 1.2: Tensor creation, element-wise addition, and inspection (Cell 4).

Executing Listing 1.2 outputs:

```
Tensor c: tensor([5., 7., 9.])
Shape of tensor c: torch.Size([3])
Data type of tensor c: torch.float32
```

1.2.3 Floating-point representations and empirical speed benchmarking

A critical engineering decision in modern AI is the choice of numerical precision. Standard scientific computing defaults to double precision (`torch.float64` or FP64), requiring 64 bits per scalar (1 sign bit, 11 exponent bits, 52 mantissa bits). In contrast, deep learning predominantly

operates with single precision (`torch.float32` or FP32: 1 sign, 8 exponent, 23 mantissa bits) or half precision (FP16 / BF16).

To observe the substantial performance differential directly, the laboratory benchmarks matrix multiplication ($\mathbf{M}_1 \times \mathbf{M}_2$) on two 1000×1000 matrices across 100 repetitions:

```
from timeit import timeit

mat_size = 1000

M1_64 = torch.randn(mat_size, mat_size, dtype=torch.float64)
M2_64 = torch.randn(mat_size, mat_size, dtype=torch.float64)

M1_32 = torch.randn(mat_size, mat_size)
M2_32 = torch.randn(mat_size, mat_size)

t64 = timeit(lambda: M1_64 @ M2_64, number=100)
t32 = timeit(lambda: M1_32 @ M2_32, number=100)

print(f"Time for matrix multiplication (float64): {t64:.4f}s")
print(f"Time for matrix multiplication (float32): {t32:.4f}s")
```

Listing 1.3: Empirical matrix multiplication benchmark comparing FP64 and FP32 (Cell 6).

Precision Format	Bit Width	Dynamic Range	Benchmark	Relative Throughput
`torch.float64` (FP64)	64 bits	$\approx 10^{\pm 308}$	0.6864 s	1.0× (Baseline)
`torch.float32` (FP32)	32 bits	$\approx 10^{\pm 38}$	0.1877 s	**3.65**× faster
`torch.float16` (FP16)	16 bits	$\approx 10^{\pm 5}$	Fast on TC	Up to 8× vs FP32
`torch.bfloat16` (BF16)	16 bits	$\approx 10^{\pm 38}$	Modern GPUs	Stable for LLMs

Table 1.1: Numerical precision comparison in deep learning. Transitioning from FP64 to FP32 yields an immediate $\approx 3.7\times$ speedup due to vector SIMD register capacity and halved memory bandwidth demands.

1.2.4 Hardware acceleration, device dispatch, and the overloaded `.to()` method

A primary advantage of PyTorch over NumPy is native support for hardware accelerators (NVIDIA CUDA GPUs and Apple Silicon MPS cores). To write hardware-agnostic code, deep learning engineers dynamically discover the execution target:

```
# Determine the optimal acceleration backend
if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")

print(f"Active execution device: {device}")
```

Listing 1.4: Hardware-agnostic device configuration and tensor dispatch.

During the laboratory lecture, the instructor emphasized the dual, polymorphic role of the `.to()` method in PyTorch:

1. **Type Casting**: Transforming numeric representations without altering physical hardware location (e.g., `x.to(torch.float16)` or `x.double()`).

2. **Hardware Placement**: Moving data and model parameters across memory boundaries (e.g., from host CPU RAM to GPU High-Bandwidth Memory via `x.to(device)`).

Exam insights: The strict homogeneous device invariant

All operands in any tensor computation (the model parameters **W**, **b**, input features **x**, and target labels **y**) **must strictly reside on the exact same physical device**.
If an input tensor on CPU is passed into a neural network situated in GPU VRAM, PyTorch immediately halts execution with a fatal runtime exception:

```
RuntimeError: Expected all tensors to be on the same device, but found at
    least two devices, cuda:0 and cpu!
```

Therefore, every robust training and inference pipeline must explicitly ensure consistent device placement before initiating the forward pass.

External deep dive: Why deep learning and LLMs mandate lower numerical precision

Standard scientific computing defaults to double precision (`torch.float64` or FP64), requiring 64 bits per scalar (1 sign bit, 11 exponent bits, 52 mantissa bits). In contrast, deep learning and natural language processing predominantly operate with single precision (`torch.float32` or FP32), half precision (`torch.float16` / `torch.bfloat16`), or quantized formats (8-bit and 4-bit).
As highlighted by the laboratory instructors, two primary engineering rationales govern this preference:

1. **Computational Throughput**: Modern GPU Tensor Cores and vector SIMD arithmetic units execute 16-bit matrix multiplications orders of magnitude faster than 64-bit doubles.

2. **GPU VRAM Capacity Bottleneck**: High-bandwidth memory (HBM) on AI accelerators is severely limited and prohibitively expensive. Every model parameter in FP32 requires 4 bytes of memory. Halving precision to FP16 or BF16 (2 bytes per parameter) instantaneously **doubles the number of parameters that fit within the same physical VRAM** with virtually zero degradation in task performance.

This capacity bottleneck provides the core motivation for model quantization, which compresses weights down to 8 bits (1 byte) or 4 bits (0.5 bytes) for efficient production deployment.

1.3 Computational graphs, autograd, and sensitivity mechanics

In classical machine learning, engineers derived analytic gradient formulas by hand and coded them manually. In modern deep learning, the framework dynamically constructs a **Directed Acyclic Graph (DAG)** of operations as forward computations execute (PyTorch's *Define-by-Run* paradigm).

1.3.1 Autograd mechanics, leaf nodes, and `requires_grad`

When a tensor is declared with `requires_grad=True`, PyTorch tracks every operation applied to it:

- **Leaf Tensors**: Tensors explicitly created by the user (e.g., weight matrices $\mathbf{W}$ and biases $\mathbf{b}$). Their gradients are accumulated in the `.grad` attribute.
- **Non-Leaf Tensors**: Intermediate results produced by operator nodes (e.g., activations $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$). Their gradients are computed dynamically during backpropagation but discarded after use to conserve memory unless `.retain_grad()` is invoked.
- **`grad_fn`**: Every non-leaf tensor stores a reference to the backward function node that created it (e.g., `AddBackward0`, `MulBackward0`).

Exam insights: `requires_grad` in training vs inference

When `requires_grad=True` is set, PyTorch allocates dedicated memory buffers for gradient tracking and builds the dynamic DAG during the forward pass.
While essential during model training, retaining `requires_grad=True` during **inference** incurs substantial, unnecessary overhead:

- It wastes GPU VRAM by storing intermediate activation histories that will never be used for backward differentiation.
- It introduces computational latency by constructing graph metadata.

Consequently, during inference or validation, engineers must always de-activate autograd graph construction using `with torch.no_grad():` or by setting `param.requires_grad = False`.

1.3.2 External authority: Andrej Karpathy's Micrograd engine

To demystify how PyTorch implements reverse-mode automatic differentiation without treating it as a black box, Andrej Karpathy developed *Micrograd*—a scalar-level autograd engine that mirrors PyTorch's C++ autograd internals in pure Python.

External deep dive: Andrej Karpathy's Micrograd: Autograd stripped to its core

In Micrograd, every number is wrapped in a lightweight `Value` object. Each object maintains:

- `data`: The forward numerical scalar.
- `grad`: The accumulated adjoint partial derivative $\partial \mathcal{L}/\partial \text{self}$.
- `_prev`: Pointers to upstream parent nodes in the computation graph.
- `_backward`: A local closure encapsulating the single-step multivariate chain rule.

```
class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    def __add__(self, other):
```

```
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), '+')
        def _backward():
            self.grad += 1.0 * out.grad
            other.grad += 1.0 * out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), '*')
        def _backward():
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def backward(self):
        # Topological sort of all nodes in the DAG
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._prev:
                    build_topo(child)
                topo.append(v)
        build_topo(self)

        # Base gradient: dLoss / dLoss = 1.0
        self.grad = 1.0
        # Traverse graph in reverse topological order
        for node in reversed(topo):
            node._backward()
```

Listing 1.5: Core structure of Andrej Karpathy's scalar autograd engine.

When calling `loss.backward()`, the engine conducts a **depth-first topological sort** of the DAG. It initializes $\frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1.0$ and propagates scalar derivatives backward through every closure. PyTorch operates on the exact same mathematical principle, scaling scalar closures to multidimensional tensor Jacobian-vector products.

Intuition & analogy: 3Blue1Brown's sensitivity view of backpropagation

Grant Sanderson (3Blue1Brown) articulates backpropagation not as abstract multivariable calculus, but as **sensitivity propagation**. Imagine giving parameter θ_1 a slight physical nudge of size ϵ. That nudge ripples forward through the graph:

$$\theta_1 \to \theta_1 + \epsilon \implies a = (\theta_1 + \epsilon)\theta_2 = a + \epsilon\theta_2. \tag{1.1}$$

The change in a wiggles the downstream product b, which shifts difference c, which finally scales loss $\mathcal{L}$ by an amount proportional to ϵ. The partial derivative $\frac{\partial \mathcal{L}}{\partial \theta_1}$ is simply the ratio of the final output wiggle to the initial parameter nudge as $\epsilon \to 0$. Reverse-mode automatic differentiation systematically traverses these interconnected sensitivity levers

backward from output to input.

1.4 Building neural models, datasets, and the canonical training loop

The laboratory transitions from primitive tensor operations to structured neural modeling.

1.4.1 Defining models with `torch.nn.Module`

In PyTorch, all neural layers and complete architectures inherit from `nn.Module`:

```
# Define a simple linear model
class SimpleLinearModel(nn.Module):
    def __init__(self, input_size, output_size):
        super(SimpleLinearModel, self).__init__()
        self.linear = nn.Linear(input_size, output_size)

    def forward(self, x):
        return self.linear(x)

# Initialize the model and configure device
model = SimpleLinearModel(1, 1)
device = torch.device("cuda" if torch.cuda.is_available() else ("mps" if
    torch.backends.mps.is_available() else "cpu"))
model = model.to(device)

print(f"Device: {device}")
print("Initial Weight:", model.linear.weight)
print("Initial Bias:", model.linear.bias)
```

Listing 1.6: Defining a single-input single-output linear model (Cells 11–18).

Intuition & analogy: Interactive classroom quiz: Parameter counting in linear layers

During the live laboratory walkthrough, the instructor posed an interactive puzzle testing students' structural understanding of linear projections:

1. **Case A ($d_{\text{in}} = 5, d_{\text{out}} = 1$)**: How many weights and biases are instantiated?
   - *Solution*: Exactly **5 weights** and **1 bias** (total = 6 parameters). Each input scalar is scaled by an individual weight, summed together, and shifted by a single scalar bias: $\hat{y} = \sum_{j=1}^{5} w_j x_j + b$.
2. **Case B ($d_{\text{in}} = 1, d_{\text{out}} = 5$)**: How many weights and biases are instantiated?
   - *Solution*: Exactly **5 weights** and **5 biases** (total = 10 parameters!).
   - *Geometric intuition*: As the instructor emphasized, moving from 1 to 5 outputs is conceptually equivalent to *parallelizing the data across 5 independent linear hyperplanes*. Each output feature neuron requires its own private scaling factor and its own translation threshold: $\hat{y}_k = w_k x + b_k$ for $k \in \{1, \dots, 5\}$.

In general, a fully connected linear layer mapping from d_{in} to d_{out} computes:

$$\mathbf{y} = \mathbf{x}\mathbf{W}^\top + \mathbf{b}, \quad \text{where } \mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}},\ \mathbf{b} \in \mathbb{R}^{d_{\text{out}}}, \tag{1.2}$$

instantiating exactly $P = d_{\text{out}} \cdot d_{\text{in}} + d_{\text{out}}$ trainable parameters.

1.4.2 Configuring criterion and optimizer

Every learning pipeline couples an objective metric with a gradient-directed stepping policy:

- **Criterion (Loss Function)**: Evaluates the magnitude of prediction error against ground-truth targets. The instructor framed the loss function as an *informed oracle*: because the loss function observes the true target y, it calculates the exact scalar error $\mathcal{L}(\hat{y}, y)$ required to steer the parameters.
  - *Continuous Regression*: Mean Squared Error (MSE):

$$\mathcal{L}_{\text{MSE}}(\hat{y}, y) = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2. \tag{1.3}$$

  - *Discrete Classification*: Multi-class Cross-Entropy Loss:

$$\mathcal{L}_{\text{CE}}(\hat{\mathbf{y}}, y) = -\log\left(\frac{e^{\hat{y}_y}}{\sum_{j=1}^{C} e^{\hat{y}_j}}\right). \tag{1.4}$$

- **Optimizer**: Dictates how parameters move in response to the backward gradient $\nabla_\theta \mathcal{L}$. While the loss function provides the *direction* of descent, the optimizer governs the *step size and trajectory dynamics*.

Exam insights: LLMs as giant classification models

The instructor highlighted an essential theoretical link bridging primitive neural networks with foundation models: *Large Language Models are, at their computational core, massive multi-class classification networks*.
Instead of classifying an image into 10 classes, an autoregressive LLM projects hidden representation $\mathbf{h}_t \in \mathbb{R}^{d_{\text{model}}}$ through an unembedding projection layer $\mathbf{W}_U \in \mathbb{R}^{V \times d_{\text{model}}}$ to output logits over a vocabulary V containing 32,000 to 128,000+ tokens, supervised by the exact same Cross-Entropy Loss formulation.

External deep dive: Optimizer dynamics: From SGD to Adam's oscillation damping

In pure Gradient Descent, updates follow:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t). \tag{1.5}$$

However, modern deep learning mandates two critical modifications:

1. **Stochastic Mini-Batching**: Evaluating the true gradient over millions of data points simultaneously is computationally impossible and exceeds GPU memory. Mini-batches (e.g., $B = 64$ or 256) provide stochastic estimates of the gradient, introducing beneficial noise that helps parameters escape shallow saddle points.
2. **Adaptive Moment Estimation (Adam)**: The instructor presented an intuitive physical view of Adam's stability. If a parameter's gradient oscillates wildly between consecutive batches (e.g., pointing positive in step t, negative in step $t+1$, and positive in step $t+2$), standard SGD wastes energy zigzagging across valley walls. Adam tracks exponentially decaying moving averages of past gradients (m_t)

and squared gradients (v_t):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2. \tag{1.6}$$

When high-frequency oscillation occurs, v_t grows relative to m_t, and the denominator $\sqrt{\hat{v}_t} + \epsilon$ automatically **dampens the effective step size**, stabilizing training without manual learning rate decay.

```
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

Listing 1.7: Configuring loss function and SGD optimizer (Cell 20).

1.4.3 Exercise 1: Synthetic regression dataset

The laboratory tasks the student with generating a synthetic dataset of $N = 2048$ points drawn from a known underlying linear relation $y = w_{\text{true}} \cdot x + b_{\text{true}} + \epsilon$, batched using `TensorDataset` and `DataLoader`.

```
# Create a dataset (synthetic linear regression dataset)
n_pts = 2048

# Ground-truth parameters to be recovered by the model
w_true = 2.0
b_true = 1.0

# Generate random inputs X ~ N(0, 1) of shape (n_pts, 1)
X = torch.randn(n_pts, 1)

# Generate targets y with additive Gaussian noise
y = w_true * X + b_true + 0.1 * torch.randn(n_pts, 1)

# Package into DataLoader
dataset = TensorDataset(X, y)
trainloader = DataLoader(dataset, batch_size=256, shuffle=True)
```

Listing 1.8: Verified solution for synthetic dataset generation (Cell 22).

External deep dive: Dataset abstractions and the mechanics of steps vs epochs

During the laboratory lecture, the instructor meticulously unpacked the roles of `TensorDataset` and `DataLoader`:

- **`TensorDataset`**: Serves as a contiguous indexing wrapper binding feature tensors $\mathbf{X} \in \mathbb{R}^{N \times 1}$ and label tensors $\mathbf{y} \in \mathbb{R}^{N \times 1}$. Slicing `dataset[i]` yields tuple $(\mathbf{x}_i, y_i)$.
- **`DataLoader`**: Manages multi-threaded mini-batch generation, memory pinning, and sample permutation:
  1. **Batch Slicing**: Subdivides the dataset into mini-batches of size $B = 256$.
  2. **Shuffling (`shuffle=True`)**: Permutes the row indices at the start of every epoch. This ensures that the model does not encounter sample pairings in identical sequence, preventing cyclical limit cycles in parameter space.

Steps vs. Epochs: A fundamental metric distinction emphasized in the course:

- An **Epoch** corresponds to one complete traversal over the entire training corpus ($N = 2048$ samples).
- A **Step** (or iteration) corresponds to a single parameter update evaluated on one mini-batch ($B = 256$).

The number of steps per epoch is given by:

$$S_{\text{epoch}} = \left\lceil \frac{N}{B} \right\rceil = \frac{2048}{256} = 8 \text{ steps per epoch.} \tag{1.7}$$

Over $E = 50$ epochs, the model performs exactly $S_{\text{total}} = 50 \times 8 = 400$ parameter update steps.

1.4.4 Exercise 1: Multi-epoch training loop with parameter tracking

To observe convergence, we implement the **5 canonical steps** of the PyTorch training loop:

1. **Gradient Reset**: `optimizer.zero_grad()` erases residual gradient accumulations from preceding mini-batches.
2. **Forward Evaluation**: `outputs = model(inputs)` traverses the neural network's forward DAG.
3. **Error Quantification**: `loss = criterion(outputs, labels)` evaluates scalar discrepancy.
4. **Adjoint Backpropagation**: `loss.backward()` traverses the DAG in reverse via autograd.
5. **Parameter Step**: `optimizer.step()` translates weights along the gradient trajectory.

```
# Training loop
num_epochs = 50

losses = []
weights = []
biases = []

model.train()
for epoch in range(num_epochs):
    running_loss = 0.0
    for inputs, labels in trainloader:
        inputs, labels = inputs.to(device), labels.to(device)

        # 1. Zero the parameter gradients from previous step
        optimizer.zero_grad()

        # 2. Forward pass: compute predictions
        outputs = model(inputs)

        # 3. Compute scalar loss
        loss = criterion(outputs, labels)

        # 4. Backward pass: compute gradients via autograd
```

```
        loss.backward()

        # 5. Optimizer step: update weights along negative gradient
        optimizer.step()

        # Track metrics at each step
        losses.append(loss.item())
        weights.append(model.linear.weight.item())
        biases.append(model.linear.bias.item())

        running_loss += loss.item()

    if (epoch + 1) % 10 == 0 or epoch == 0:
        print(f"Epoch [{epoch+1}/{num_epochs}], Loss:
            {running_loss/len(trainloader):.4f}")
```

Listing 1.9: Verified solution for the 50-epoch training loop with metric tracking (Cell 24).

```
Epoch [1/50], Loss: 2.2321
Epoch [10/50], Loss: 0.1246
Epoch [20/50], Loss: 0.0140
Epoch [30/50], Loss: 0.0099
Epoch [40/50], Loss: 0.0097
Epoch [50/50], Loss: 0.0097
Learned weights: tensor([[2.0000]])
Learned biases: tensor([0.9988])
```

The model successfully converges from arbitrary random initialization to recover the true underlying slope $w \to 2.0000$ and intercept $b \to 0.9988$, matching the 3-panel visualization in Figure 1.2.

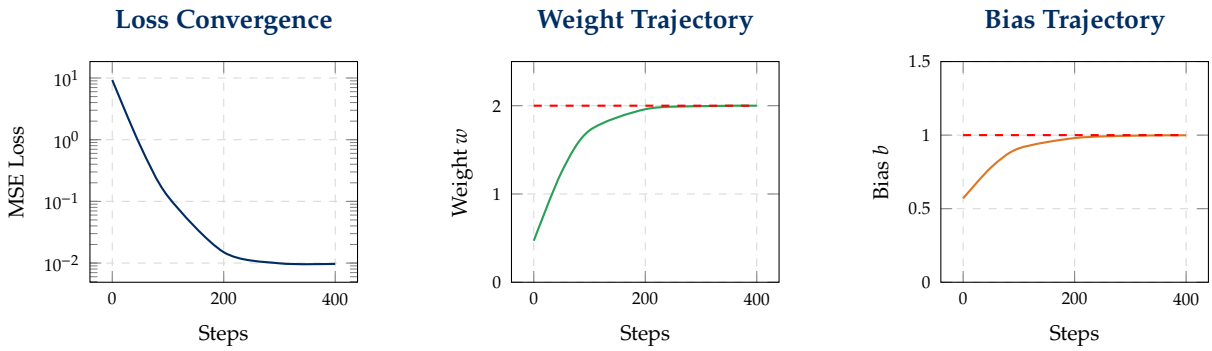


Figure 1.2: Visualizing optimization dynamics across 50 epochs (Cell 25). The mean squared error decays logarithmically towards the irreducible noise floor ($\sigma^2 = 0.01$), while weight w asymptotically reaches 2.0 and bias b reaches 1.0.

1.5 Computer vision pipeline and data augmentation (CIFAR-10)

Moving beyond 1D synthetic scalars, Section 2 of the laboratory introduces the **CIFAR-10** benchmark (60,000 32×32 color images across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck).

1.5.1 Channel ordering: PyTorch vs Matplotlib

PyTorch standardizes on the **Channel-First** format $[B, C, H, W]$, where $C = 3$ color channels (Red, Green, Blue) precede height $H = 32$ and width $W = 32$. However, Matplotlib and visualization libraries expect the **Channel-Last** format $[H, W, C]$. To render a PyTorch image tensor with `plt.imshow()`, one must invoke:

$$\mathbf{x}_{\text{plot}} = \mathbf{x}_{\text{torch}}.\text{permute}(1, 2, 0). \tag{1.8}$$

1.5.2 Exercise 2: Computing dataset mean and standard deviation from scratch

Standardization accelerates gradient descent by ensuring zero-centered inputs with unit variance across all feature channels:

$$\tilde{x}_{c,i,j} = \frac{x_{c,i,j} - \mu_c}{\sigma_c} \quad \text{for } c \in \{R, G, B\}. \tag{1.9}$$

In Cell 33, students are tasked with computing μ_c and σ_c across all 50,000 training images scaled to $[0, 1]$:

```
# Download initial dataset to compute raw statistics
import os
dataset_dir = os.path.expanduser('~/data')

trainset = torchvision.datasets.CIFAR10(
    root=dataset_dir, train=True, download=True, transform=transforms.ToTensor()
)

# trainset.data is a numpy array of shape (50000, 32, 32, 3) with uint8 values [0,
    255]
# Scale to [0, 1] floating-point range
scaled_data = trainset.data / 255.0

# Compute mean and standard deviation across image batches, heights, and widths (axes
    0, 1, 2)
images_mean = tuple(scaled_data.mean(axis=(0, 1, 2)).tolist())
images_std = tuple(scaled_data.std(axis=(0, 1, 2)).tolist())

print(f"Calculated Mean (R, G, B): {images_mean}")
print(f"Calculated Std  (R, G, B): {images_std}")
```

Listing 1.10: Verified solution for calculating dataset mean and standard deviation per channel (Cell 33).

The resulting values match the canonical CIFAR-10 population statistics:

$$\boldsymbol{\mu} = (0.4914, 0.4822, 0.4465), \quad \boldsymbol{\sigma} = (0.2470, 0.2435, 0.2616). \tag{1.10}$$

1.5.3 Data augmentation and visualization pipeline

Data augmentation synthetically expands the training distribution by applying label-preserving geometric perturbations. In Cell 37, the laboratory constructs a visualization pipeline applying `RandomHorizontalFlip` and `RandomCrop` with reflection padding:

```
viz_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    # Reflect border pixels to handle crop margins
    transforms.RandomCrop(32, padding_mode='reflect', padding=5)
])
```

```

# Extract a sample image tensor and convert to PIL format
images, labels = next(iter(trainloader))
im = transforms.ToPILImage()(images[10])
n_transforms = 9

fig, ax = plt.subplots(3, 3, figsize=(4, 4))

# Slot 0: Original unaugmented image
ax[0, 0].imshow(im)
ax[0, 0].set_title('Original', fontsize=8)
ax[0, 0].axis('off')

# Slots 1-8: Stochastic variations generated by the transform
for i in range(1, n_transforms):
    r, c = divmod(i, 3)
    ax[r, c].imshow(viz_transform(im))
    ax[r, c].set_title(f'Aug {i}', fontsize=8)
    ax[r, c].axis('off')

plt.tight_layout()
```

Listing 1.11: Verified solution for data augmentation visualization (Cell 37).

1.6 Multi-layer perceptron for classification (`SimpleNN`)

In Section 2.0, the laboratory constructs a Multi-Layer Perceptron (MLP) for 10-class image classification.

1.6.1 Mathematical formulation of the architecture

The network maps a 3D image tensor into a flattened vector, passes it through two hidden layers with Rectified Linear Unit (ReLU) activations, and outputs unnormalized class logits:

$$\mathbf{x}_{\text{flat}} = \text{vec}(\mathbf{x}) \in \mathbb{R}^{32\times32\times3} = \mathbb{R}^{3072}, \tag{1.11}$$

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1\mathbf{x}_{\text{flat}} + \mathbf{b}_1) \in \mathbb{R}^{512}, \qquad \mathbf{W}_1 \in \mathbb{R}^{512\times3072},\ \mathbf{b}_1 \in \mathbb{R}^{512}, \tag{1.12}$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2) \in \mathbb{R}^{256}, \qquad \mathbf{W}_2 \in \mathbb{R}^{256\times512},\ \mathbf{b}_2 \in \mathbb{R}^{256}, \tag{1.13}$$

$$\hat{\mathbf{y}} = \mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3 \in \mathbb{R}^{10}, \qquad \mathbf{W}_3 \in \mathbb{R}^{10\times256},\ \mathbf{b}_3 \in \mathbb{R}^{10}. \tag{1.14}$$

Figure 1.3 illustrates the end-to-end architectural topology of `SimpleNN`, visualizing the tensor dimensions, fully connected weight matrices, non-linear ReLU activations, and reverse-mode gradient flow.

1.6.2 Exercise: Implementing `SimpleNN`

```
class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        # Linear layer 1: 3072 input features -> 512 hidden features
        self.fc1 = nn.Linear(32 * 32 * 3, 512)
        # Linear layer 2: 512 hidden features -> 256 hidden features
        self.fc2 = nn.Linear(512, 256)
        # Output layer: 256 hidden features -> 10 class logits
        self.fc3 = nn.Linear(256, 10)
```

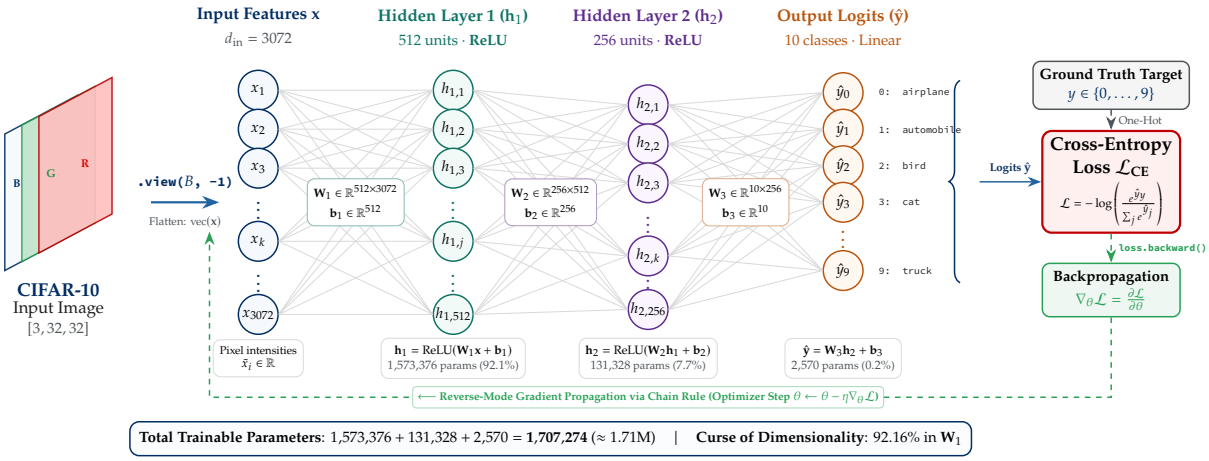


Figure 1.3: Architectural topology of the `SimpleNN` Multi-Layer Perceptron (MLP) for CIFAR-10 classification (Section 1.6.1). An input color image $[3 \times 32 \times 32]$ is flattened into feature vector $\mathbf{x} \in \mathbb{R}^{3072}$, sequentially propagated through two hidden fully-connected layers with non-linear ReLU activations (512 and 256 units), and projected into 10 unnormalized class logits $\hat{\mathbf{y}} \in \mathbb{R}^{10}$. Training is supervised by Cross-Entropy Loss with gradients propagated in reverse mode.

```
    def forward(self, x):
        # Flatten input: preserve batch dimension, collapse (C, H, W)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        # Logits output (no activation needed; CrossEntropyLoss applies Softmax
            internally)
        x = self.fc3(x)
        return x
```

Listing 1.12: Verified implementation of `SimpleNN` architecture (Cell 42).

1.6.3 Evaluation function and gradient deactivation

Validation requires disabling parameter updates to prevent memory leaks and ensure deterministic evaluation:

```
def val_model(model, testloader):
    model.eval()  # Switch to evaluation mode (deactivates dropout/batchnorm training
        behavior)
    correct = 0
    total = 0
    with torch.no_grad():  # Disable autograd DAG construction to conserve memory
        for inputs, labels in testloader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
    print(f"Accuracy of the network on test images: {accuracy:.2f}%")
```

Listing 1.13: Validation routine with `model.eval()` and `torch.no_grad()` (Cell 48).

Prior to training, random guessing on a 10-class dataset yields an accuracy of $\approx 10.0\%$. After 5 epochs of training via Listing 1.9, the multi-layer perceptron achieves approximately 47.5% accuracy.

1.7 Transfer learning and parameter freezing with ResNet-18

While training an MLP from scratch yields $\approx 47\%$ accuracy on CIFAR-10, modern engineering relies on **transfer learning**: reusing representations acquired by deep models pretrained on massive datasets (such as ImageNet's 1.2 million images).

1.7.1 Deep residual learning (ResNet)

Introduced in [1], Deep Residual Networks solve the vanishing gradient problem in networks with dozens of layers by introducing **identity shortcut connections**:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{\mathbf{W}_i\}) + \mathbf{x}. \tag{1.15}$$

During backpropagation, the additive residual connection ensures gradients flow unimpeded directly to early layers:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left(\frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \mathbf{I} \right). \tag{1.16}$$

Even if the learned Jacobian $\frac{\partial \mathcal{F}}{\partial \mathbf{x}}$ vanishes, the identity term $\mathbf{I}$ guarantees an uninterrupted gradient conduit.

1.7.2 Exercise: Adapting pretrained ResNet-18 for CIFAR-10

The base `models.resnet18` has an output fully connected layer (`fc`) sized for 1000 ImageNet classes. In Cell 65, the student replaces this head with a new 10-class linear projection. Figure 1.4 illustrates the transfer learning pipeline, contrasting the pre-trained ImageNet source model against the adapted CIFAR-10 downstream architecture.

```
from torchvision import models

# Load ResNet-18 with ImageNet weights
pretrained_model = models.resnet18(weights='DEFAULT')

# Access original input feature dimension (512) and replace head
pretrained_model.fc = nn.Linear(pretrained_model.fc.in_features, 10)

# Transfer model to active device
pretrained_model = pretrained_model.to(device)
```

Listing 1.14: Loading pretrained ResNet-18 and replacing the classifier head (Cell 65).

Fine-tuning this model for 5 epochs elevates validation accuracy from 47.5% to over **82.4**%.

1.7.3 Exercise: Transfer to CIFAR-100 and parameter freezing

In Section 2.3, the challenge shifts to fine-tuning the model for **CIFAR-100** (100 fine-grained classes). To prevent overwriting the valuable pretrained visual feature extractors and drastically accelerate training, we **freeze all backbone layers** and only train the newly attached classification head:

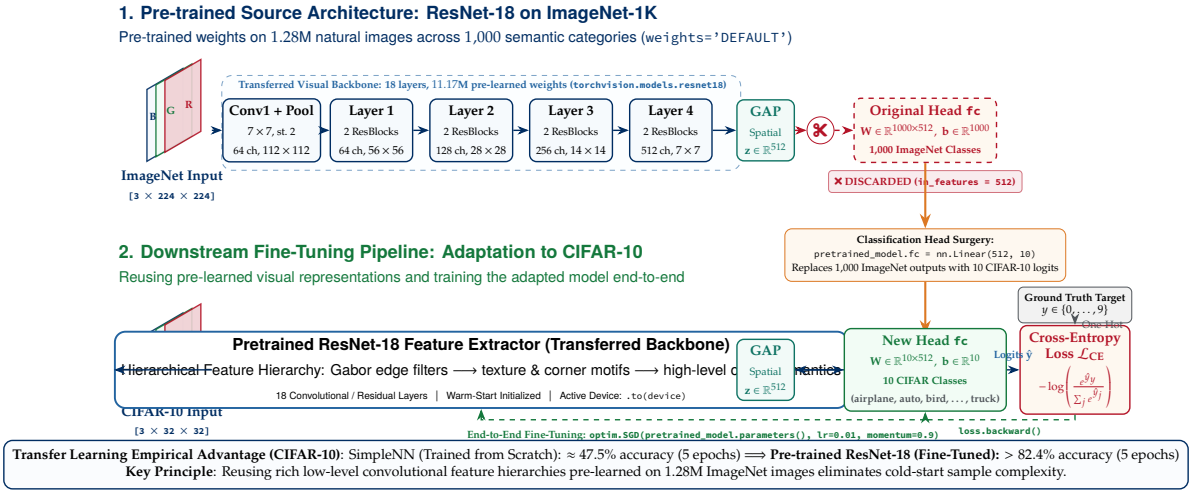


Figure 1.4: Architectural topology of transfer learning and fine-tuning with ResNet-18 on CIFAR-10 (Section 1.7.2). (Top) The source ResNet-18 model pre-trained on ImageNet-1K (1.28M images, 1,000 categories) features 18 residual convolutional layers and an original 1,000-class classification head. (Middle) Classification head surgery replaces `pretrained_model.fc` with a new linear projection $\mathbf{W} \in \mathbb{R}^{10\times512}$, $\mathbf{b} \in \mathbb{R}^{10}$. (Bottom) End-to-end downstream fine-tuning on CIFAR-10 ($[3 \times 32 \times 32]$), propagating gradients through both the new head and the pre-trained feature extractor, boosting classification accuracy from 47.5% (MLP from scratch) to > 82.4% in just 5 epochs.

```python
# Modify classification head for 100 classes
model = pretrained_model
model.fc = nn.Linear(model.fc.in_features, 100)
model = model.to(device)

# Step 1: Freeze all layers across the network
for param in model.parameters():
    param.requires_grad = False

# Step 2: Unfreeze ONLY the classification head parameters
for param in model.fc.parameters():
    param.requires_grad = True

# Optimizer tracks ONLY the head parameters
optimizer = optim.SGD(model.fc.parameters(), lr=0.01, momentum=0.9, weight_decay=5e-4)
```

Listing 1.15: Freezing backbone parameters and unfreezing the classification head (Cells 72–75).

Configuration	Total Parameters	Trainable Parameters	Trainable Ratio
Full ResNet-18 Fine-Tuning	11,176,512	11,176,512	100.0%
Frozen Backbone + Head Only	11,227,584	**51,300**	**0.457**%

Table 1.2: Parameter efficiency comparison under layer freezing. By training only the newly attached classification head (`model.fc`), the number of active gradients drops by over 99.5%, serving as a direct conceptual precursor to LoRA (Laboratory 05).

1.7.4 The foundation model paradigm: Bridging ResNet in vision and BERT in NLP

During the laboratory lecture, the instructor established an indispensable pedagogical bridge connecting convolutional vision backbones with large language models. In industrial machine learning, the era of training massive architectures from scratch on modest consumer hardware has drawn to a close:

- **The Economics of Pretraining**: Pretraining foundation models (such as BERT, RoBERTa, LLaMA, or deep vision backbones) demands millions of dollars in compute, warehouse-scale GPU clusters, and massive data ingestion pipelines. More than 99% of practitioners and industrial teams cannot pretrain foundation models from scratch.
- **Latent Space Representations**: Pretrained foundation models arrive equipped with non-random, semantically structured weights. In vision, ResNet maps raw pixels into rich hierarchical visual features (edges → textures → object contours). In NLP, an encoder like BERT maps raw token sequences into structured latent embeddings $\mathbf{z} \in \mathbb{R}^{d_{\text{model}}}$ capturing syntactic grammar, context, and semantic co-occurrence.

Strategy	Training From Scratch	Full Fine-Tuning	Linear Probing (Freezing)
Weight Initialization	Random Gaussian / Xavier	Pretrained foundation weights	Pretrained foundation weights
Active Gradients	All layers (100%)	All layers (100%)	Output head only (< 0.5%)
Memory Footprint	High (full optimizer states)	Very High (full backward graph)	Minimal (activations frozen)
Training Speed	Very Slow	Moderate	Very Fast (few epochs)
Data Requirement	Massive dataset required	Modest downstream dataset	Small downstream dataset
Convergence Risk	High risk of divergence/overfit	Stable, high accuracy	Fast, competitive accuracy
Downstream Example	Training ResNet from zero	Fine-tuning BERT end-to-end	Frozen BERT + classification head

Table 1.3: Systematic comparison of downstream adaptation strategies across computer vision and natural language processing, as presented during the laboratory lecture.

Intuition & analogy: From parameter freezing to PEFT and LoRA

Notice the profound connection to Large Language Models: freezing the backbone and training a lightweight linear adapter on top (51,300 parameters vs. 11.2M) achieves nearly identical accuracy in a fraction of the wall-clock time while completely eliminating gradient computations across deep residual blocks.
This exact mathematical philosophy underpins **Parameter-Efficient Fine-Tuning (PEFT)** and **Low-Rank Adaptation (LoRA)** in modern LLMs: instead of fine-tuning all 7B or 70B parameters of a foundation model, engineers freeze the transformer weights and update only low-rank adapter matrices, drastically lowering VRAM thresholds and democratizing model customization.

1.8 Exercise 3 in-depth: analytical derivation and autograd DAG trace

The final exercise of the laboratory (Cells 80–84) analyzes automatic differentiation on a composite mathematical function, reconciling analytical calculus with PyTorch's autograd engine.

1.8.1 Mathematical problem definition

Given input $x = 1.0$, target $y = 3.0$, and parameters $\theta_1 = 2.0$, $\theta_2 = 3.0$, we define the predicted value $\hat{y}$ and quadratic loss $\mathcal{L}$:

$$\hat{y} = \theta_1 \theta_2 x, \quad \mathcal{L}(\theta_1, \theta_2) = (\theta_1 \theta_2 x - y)^2. \tag{1.17}$$

1.8.2 Analytical derivation of partial derivatives

We decompose the forward pass into elementary atomic operations:

$$a = \theta_1 \theta_2, \tag{1.18}$$
$$b = a \cdot x, \tag{1.19}$$
$$c = b - y, \tag{1.20}$$
$$\mathcal{L} = c^2. \tag{1.21}$$

Applying the multivariate chain rule in reverse:

$$\frac{\partial \mathcal{L}}{\partial c} = 2c = 2(b - y) = 2(\theta_1 \theta_2 x - y), \tag{1.22}$$
$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial c} \frac{\partial c}{\partial b} = 2c \cdot 1 = 2c, \tag{1.23}$$
$$\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial b} \frac{\partial b}{\partial a} = 2c \cdot x = 2cx. \tag{1.24}$$

Now computing the parameter sensitivities:

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_1} = (2cx) \cdot \theta_2 = 2(\theta_1 \theta_2 x - y)\theta_2 x, \tag{1.25}$$
$$\frac{\partial \mathcal{L}}{\partial \theta_2} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_2} = (2cx) \cdot \theta_1 = 2(\theta_1 \theta_2 x - y)\theta_1 x. \tag{1.26}$$

1.8.3 Numerical evaluation

Substituting the problem's values $\theta_1 = 2.0, \theta_2 = 3.0, x = 1.0, y = 3.0$:

$$a = 2.0 \times 3.0 = 6.0, \tag{1.27}$$
$$b = 6.0 \times 1.0 = 6.0, \tag{1.28}$$
$$c = 6.0 - 3.0 = 3.0, \tag{1.29}$$
$$\mathcal{L} = 3.0^2 = \mathbf{9.0}. \tag{1.30}$$

Evaluating the gradients:

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = 2(3.0) \times 3.0 \times 1.0 = \mathbf{18.0}, \tag{1.31}$$
$$\frac{\partial \mathcal{L}}{\partial \theta_2} = 2(3.0) \times 2.0 \times 1.0 = \mathbf{12.0}. \tag{1.32}$$

1.8.4 PyTorch implementation and computational graph visualization

```python
import torch
from torchviz import make_dot

# Define parameter and input tensors with gradient tracking
theta1 = torch.tensor(2.0, requires_grad=True)
theta2 = torch.tensor(3.0, requires_grad=True)
x = torch.tensor(1.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)

# Forward pass: compute scalar loss
loss = (theta1 * theta2 * x - y)**2

# Visualize computational graph
dot = make_dot(loss, params={'theta1': theta1, 'theta2': theta2, 'x': x, 'y': y})

# Execute reverse-mode automatic differentiation
loss.backward()

print("Computed Loss:", loss.item())
print("dL/d(theta1):", theta1.grad.item())
print("dL/d(theta2):", theta2.grad.item())
```

Listing 1.16: Verified solution for Exercise 3 autograd calculation and visualization (Cells 83–84).

```
Computed Loss: 9.0
dL/d(theta1): 18.0
dL/d(theta2): 12.0
```

PyTorch's automated backward traversal produces exactly 18.0 and 12.0, perfectly confirming our analytic derivation. As highlighted during the laboratory lecture, invoking `torchviz.make_dot` exposes PyTorch's internal C++ autograd backward DAG:

- **Accumulation Nodes**: Leaf variables declared with `requires_grad=True` (θ_1, θ_2, x, y) terminate at `AccumulateGrad` nodes that accumulate incoming adjoint gradients into each tensor's `.grad` buffer.
- **Backward Operators**: Non-leaf operation nodes (such as `MulBackward0` or `SubBackward0`) represent the local multivariate derivative closures executed when `loss.backward()` is triggered.

Figure 1.5 provides a vector visualization of this exact Directed Acyclic Graph, contrasting forward computation with reverse-mode adjoint propagation.

1.9 Laboratory summary and engineering checklist

Table 1.4 synthesizes the core PyTorch modules, classes, and design patterns mastered in Laboratory 01.

1.9.1 Bridge to Laboratory 02

In Laboratory 01, we established the fundamental tensor mechanisms of deep learning, progressing from scalar regression to convolutional transfer learning. In **Laboratory 02**, we transition from continuous computer vision pixels to discrete natural language text. We will explore how subword tokenizers (Hugging Face `AutoTokenizer`) map text into discrete token IDs, examine the

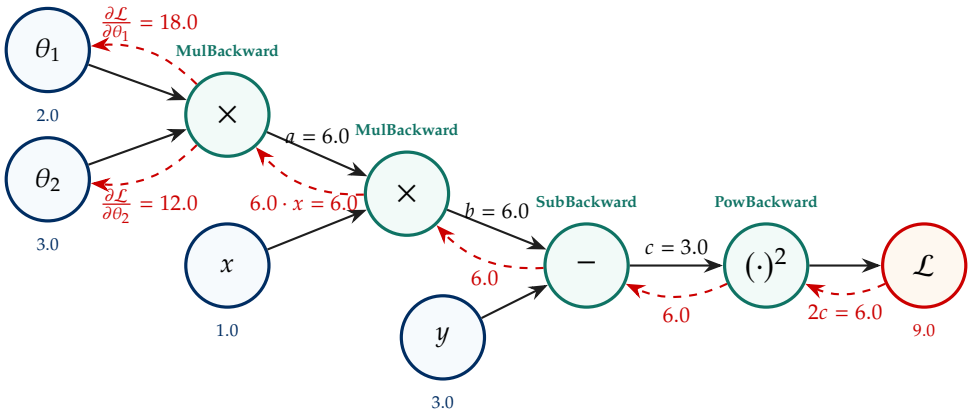


Figure 1.5: Vector rendering of the computational graph for Exercise 3 (reproducing the notebook's `computational_graph.png` and `backpropagation.png`). Black solid paths trace forward evaluation; red dashed arrows trace reverse-mode adjoint gradient propagation via the chain rule.

PyTorch API / Component	Engineering Function & Best Practice
`torch.tensor(..., requires_grad=True)`	Leaf node instantiation with autograd sensitivity tracking.
`torch.randn(shape)`	Gaussian initialization; prefer FP32/BF16 over FP64 for throughput.
`nn.Module`	Object-oriented base class for all neural layers and architectures.
`nn.Linear(in_features, out_features)`	Affine matrix transformation $\mathbf{Wx} + \mathbf{b}$ with automatic parameter registration.
`TensorDataset(X, y)`	Packs input and label tensors into a unified iterable indexing structure.
`DataLoader(dataset, batch_size, shuffle)`	Provides minibatching, shuffling, and multi-threaded parallel worker loading.
`optimizer.zero_grad()`	Resets parameter gradients to zero before backpropagation to prevent accumulation.
`loss.backward()`	Executes reverse-mode DAG topological traversal and accumulates partial derivatives.
`optimizer.step()`	Updates model parameters along the negative gradient vector.
`model.eval()`	Sets evaluation mode; freezes stochastic layers (e.g., dropout and batchnorm).
`torch.no_grad()`	Context manager deactivating the autograd graph engine to conserve VRAM.
`param.requires_grad = False`	Parameter freezing; isolates feature backbones during transfer learning.

Table 1.4: Summary of foundational PyTorch APIs mastered in Laboratory 01.

encoder-decoder architecture of **T5**, and construct the **Scaled Dot-Product Attention mechanism** ($QK^\top/\sqrt{d_k}$) from scratch.

CHAPTER 2

Laboratory 02: Hugging Face ecosystem, encoder-decoder models, and attention mechanisms

2.1 Laboratory overview and macro perspective in LLM systems

In Laboratory 01, we established the fundamental computational substrate of modern deep learning: multidimensional tensors, strided memory layouts, automatic differentiation (autograd) via dynamically constructed Directed Acyclic Graphs (DAGs), and the iterative gradient descent optimization loop. While these primitives allow us to construct arbitrary multilayer perceptrons and convolutional backbones, language modeling introduces a fundamentally different challenge: processing variable-length discrete symbolic sequences where the meaning of any individual token depends dynamically on its contextual relationship with every other token in the sequence.

Laboratory 02 bridges this foundational gap by transitioning from atomic tensor algebra to production-grade **Transformer architectures** and the **Attention Mechanism**—the foundational innovation introduced by Vaswani et al. [2] that powers all modern Large Language Models.

2.1.1 From tensor primitives to attention-driven foundation models

To appreciate why the attention mechanism revolutionized deep learning, consider the limitations of the computational layers explored in Laboratory 01:

1. **Linear (Dense) Layers**: A standard linear layer $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$ computes static affine transformations with fixed input dimensionality. It possesses no inherent mechanism to handle variable-length sequences or to model position-dependent interactions without concatenating all tokens into an unwieldy, parameter-heavy vector.

2. **Recurrent Neural Networks (RNNs/LSTMs)**: While historically deployed for sequential text, recurrent models process tokens sequentially ($\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$). This sequential data dependency creates an insurmountable training bottleneck: forward passes and backpropagation through time (BPTT) cannot be parallelized across sequence lengths T on massively parallel hardware accelerators (GPUs/TPUs). Furthermore, recurrent hidden states suffer from information bottlenecks and vanishing/exploding gradients when bridging long-range token dependencies [3, 4].

3. **Attention Mechanisms**: Attention circumvents both limitations by computing direct, pairwise similarity scores between all tokens in a sequence simultaneously. This enables $O(1)$ path length between any two arbitrary positions regardless of sequential distance, while executing entirely as batched, highly optimized matrix multiplications ($\mathbf{Q}\mathbf{K}^\top\mathbf{V}$) perfectly matched to modern tensor cores [2, 5].

2.1.2 Taxonomy of Transformer architectures: Encoder-decoder, causal decoder, and masked encoder

Not all Transformer models are structured identically. Depending on the sequence-to-sequence transformation required, the literature divides Transformer topologies into three primary structural archetypes, summarized in Table 2.1.

Architecture	Attention Masking	Canonical Models	Target Applications
Encoder-Only	Full bidirectional (unmasked) across all sequence tokens	BERT [6], RoBERTa, CodeBERT [7]	Token classification, masked language modeling, sentence embeddings, code search.
Decoder-Only	Causal triangular mask (tokens attend strictly to past positions)	GPT-2/3/4 [8, 9], LLaMA [10], Mistral	Autoregressive text generation, open-ended code synthesis, conversational chat.
Encoder-Decoder	Bidirectional in encoder, causal in decoder, cross-attention interconnect	T5 [11], BART, Original Transformer [2]	Text-to-text transformation, neural machine translation, code summarization, refactoring.

Table 2.1: Taxonomy of Transformer architectural paradigms across attention topology and downstream tasks.

This laboratory directly investigates the **Encoder-Decoder paradigm** using Google's T5 (Text-to-Text Transfer Transformer) [11], before isolating and building the underlying **Scaled Dot-Product Attention mechanism** from first principles in pure PyTorch.

2.1.3 Pedagogical roadmap and learning outcomes

Laboratory 02 is divided into two deeply complementary computational modules:

1. **Module 1: Hugging Face Ecosystem and T5 Architecture Exploration (`text-01-t5.ipynb`)**:
    - Ingesting and tokenizing text using the Hugging Face `transformers` library.
    - Mastering SentencePiece subword tokenization, vocabulary structures, and the whitespace-delimited underscore prefix (_).
    - Analyzing sequence batching, zero-padding, attention masking, and GPU memory overheads.
    - Inspecting T5's internal parameter organization, multi-head dimensions ($768 = 12 \times 64$), and weight-sharing mechanisms across embeddings and linear heads.
    - Geometric inspection of latent embeddings via Principal Component Analysis (PCA) and Cosine Similarity matrices.
    - Implementing the manual autoregressive generation loop step-by-step, contrasting it with optimized KV-cached generation (`model.generate()`).
    - Extracting, dissecting, and visualizing internal cross-attention tensors across all 12 layers and attention heads.
2. **Module 2: Scaled Dot-Product Attention from Scratch on Synthetic Sequences (`text-02-attention.ipy`**
    - Formulating discontinuous periodic time series prediction as a sequence regression task.

- Constructing custom PyTorch `Dataset` and `DataLoader` pipelines with randomized jump discontinuities.
- Implementing custom `ScaledDotProductAttention` and multi-parameter `AttentionHead` modules from scratch.
- Mathematically deriving the $\frac{1}{\sqrt{d_k}}$ scaling factor and its variance-stabilization proof.
- Conducting a rigorous empirical benchmark comparing **Relative Positional Encodings** (Shaw et al. [12] / T5 style) against **Absolute Sinusoidal Positional Encodings** (Vaswani et al. [2]).
- Proving why relative positional representations converge to near-zero loss (0.0363) on periodic invariants where absolute encodings completely stagnate (5.4174).

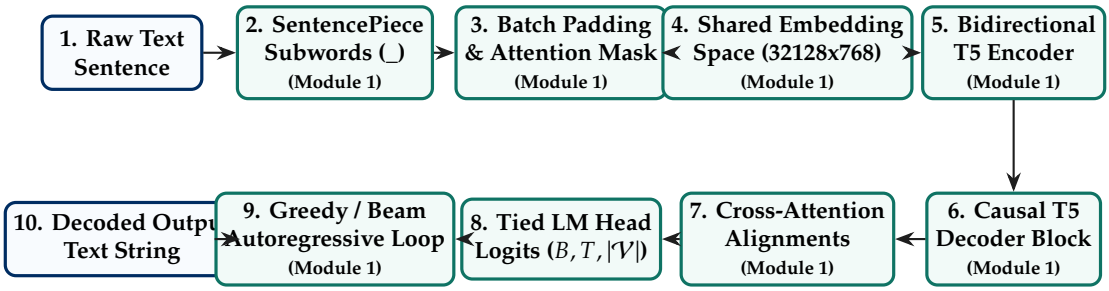


Figure 2.1: End-to-end dataflow in the Hugging Face T5 Sequence-to-Sequence pipeline explored throughout Laboratory 02.

2.2 Tokenization pipelines, subwords, and batch padding mechanics

Large Language Models do not process raw ASCII or UTF-8 text strings directly. Instead, text must undergo deterministic conversion into a discrete sequence of integer indices drawn from a predefined vocabulary $\mathcal{V}$, a process termed **tokenization**.

2.2.1 The role of tokenizers in language models

In early Natural Language Processing pipelines, tokenization relied on whitespace splitting or heuristic rule-based stemmers. These approaches faced two catastrophic failure modes:

1. **Out-Of-Vocabulary (OOV) Explosion**: Natural languages exhibit virtually unbounded vocabularies due to morphological compounding, slang, spelling variations, and programming language identifiers (`getUserById`, `is_valid`). Word-level vocabularies quickly expand into millions of tokens while still failing on unseen words.

2. **Character-Level Dilution**: While character-level models eliminate OOV tokens, they drastically inflate sequence lengths (T), causing self-attention computational complexity ($O(T^2)$) and memory consumption to explode, while diluting semantic signal across individual characters.

Modern foundational models resolve this dilemma via **Subword Tokenization algorithms** (such as Byte-Pair Encoding (BPE) [13], WordPiece, and SentencePiece Unigram [14]). Frequent words remain single atomic tokens, while rare, compound, or domain-specific words are decomposed into compact subword fragments.

2.2.2 SentencePiece and the whitespace-delimited underscore prefix

The T5 model utilizes Google's **SentencePiece** tokenizer library with a Unigram language model vocabulary of 32,128 tokens [11, 14]. Unlike traditional tokenizers that treat whitespace as an external delimiter discarded before tokenization, SentencePiece treats whitespace as a first-class character, represented by a special meta-symbol: the unicode underscore prefix _ (specifically U+2581 _).

> **Intuition & analogy: The Underscore Prefix as a Reversible Word Boundary**
>
> Why does SentencePiece prepend an underscore _ to tokens instead of storing separate space characters?
> In standard tokenization, converting `"hello world"` into tokens and then decoding might lose exact whitespace information (e.g., whether there was a single space, multiple spaces, tabs, or no space before punctuation). By explicitly prepending _ to any subword that was preceded by whitespace, tokenization becomes strictly **lossless and reversible** (*detokenization* requires only concatenating tokens and replacing _ with a space).
> Crucially, as emphasized during the course lecture by teaching assistant Claudio Savelli, the underscore also alters the token's semantic identity:
>
> - `"_pre"`: Represents the token `pre` appearing at the beginning of a word or after whitespace (e.g., `"_pre-requisite"`).
> - `"pre"`: Represents the identical character sequence occurring inside a word (e.g., within `"com_pre_hend"`).
>
> The neural network assigns entirely distinct embedding vectors to `"_pre"` and `"pre"`, allowing it to contextualize morphological prefixes differently from internal syllables.

2.2.3 Batching, padding, and attention masks

Deep learning models achieve their computational throughput by evaluating multiple sequences simultaneously in a batch ($B > 1$). Because GPU kernels (such as cuBLAS GEMM) execute on rectangular multidimensional tensors of fixed shape $[B, T, d_{\text{model}}]$, all sequences in a given batch must possess identical length T.

When batching sentences of varying natural lengths, two operations are required:

1. **Zero-Padding (`padding=True`)**: The longest sequence in the batch defines the maximum length $T_{\max}$. All shorter sequences are padded with a designated pad token (`pad_token_id = 0` in T5) until they reach $T_{\max}$.
2. **Attention Masking**: Adding dummy pad tokens introduces a severe correctness hazard: self-attention would ordinarily compute query-key dot products between legitimate semantic tokens and meaningless pad tokens, polluting representations and skewing softmax distributions. To prevent this, the tokenizer produces a binary **Attention Mask** tensor $\mathbf{M} \in \{0, 1\}^{B \times T}$:

$$M_{b,t} = \begin{cases} 1, & \text{if position } t \text{ is an authentic input token,} \\ 0, & \text{if position } t \text{ is a padded token.} \end{cases} \tag{2.1}$$

   In the attention mechanism, positions where $M_{b,t} = 0$ have $-\infty$ added to their pre-softmax attention logits, ensuring that $\text{softmax}(-\infty) = 0$ and completely nullifying any information flow to or from padded positions.

Exam insights: Memory Saturation and Dynamic Bucketing

Padding introduces a critical GPU memory and computational inefficiency. If a batch contains 31 short sentences of length 10 and one outlier sentence of length 512, all 31 short sequences are padded to length 512! Over 98% of the allocated tensor elements represent non-informative zeros, saturating GPU VRAM and performing wasteful $O(T_{\text{max}}^2)$ attention calculations.
In production LLM training pipelines (and explored further in Laboratory 03), engineers counteract this via:

1. **Truncation (`truncation=True, max_length=K`)**: Hard-capping sequence length to prevent single outliers from blowing up batch shapes.
2. **Dynamic Batching / Length Bucketing**: Sorting dataset sequences by token length prior to batch construction, ensuring that sequences within each batch possess nearly identical lengths, minimizing zero-padding overhead.

2.2.4 Verified code: Tokenization, padding, and batch decoding

Listing 2.1 displays the verified implementation from Cell 2 of `text-01-t5.ipynb`, loading the pretrained T5 tokenizer and inspecting its class hierarchy.

```
from transformers import AutoTokenizer

model_name = "google-t5/t5-base"

# Instantiate tokenizer from Hugging Face Hub
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Verified check: print the type of the tokenizer
print(type(tokenizer))
# Output: <class 'transformers.models.t5.tokenization_t5_fast.T5TokenizerFast'>

# Tokenize a basic sentence
sentence = "hello, this is a sentence!"
tokens = tokenizer(sentence)
print(tokens)
```

Listing 2.1: Loading T5 tokenizer and verifying class type (Cells 2–5).

Executing this cell produces the token dictionary:

```
{'input_ids': [21820, 6, 44, 19, 3, 9, 7142, 55, 1], 'attention_mask': [1, 1, 1, 1,
    1, 1, 1, 1, 1]}
```

Notice the trailing token ID `1`. In T5, token ID `1` represents the End-Of-Sequence sentinel (`</s>`), explicitly appended by the tokenizer to mark sequence completion.

We can inspect the vocabulary size and probe the internal token-to-ID mapping using Cells 10–28 of the notebook:

```
import random

vocabulary = tokenizer.get_vocab()
print("Total vocabulary size:", len(vocabulary)) # Output: 32128

# Reverse mapping: token_id -> string
reverse_vocab = {v: k for k, v in vocabulary.items()}
```

```

# Inspect special tokens
print("Special tokens:", tokenizer.eos_token, tokenizer.pad_token,
    tokenizer.bos_token)
# Output: </s> <pad> None

print("Special token IDs:", tokenizer.eos_token_id, tokenizer.pad_token_id,
    tokenizer.bos_token_id)
# Output: 1 0 None

# Inspect subword tokenization with whitespace variations
print(tokenizer.tokenize("hello, this is a sentence!"))
# Output: [' hello', ',', ' this', ' is', ' ', 'a', ' sentence', '!']

print(tokenizer.tokenize("hello    ,world"))
# Output: [' hello', ' ', ',', 'world']
print(tokenizer.tokenize("hello    , world"))
# Output: [' hello', ' ', ',', ' world']
```

Listing 2.2: Probing vocabulary structure and special token IDs (Cells 10–28).

In the lecture walkthrough, TA Claudio Savelli highlighted that SentencePiece collapses multiple consecutive spaces into a single _ meta-token followed by standard subwords, demonstrating the whitespace-normalizing properties of the Unigram tokenizer.

Next, Listing 2.3 demonstrates the verified solution for batch tokenization with automatic padding, contrasting standard list outputs with PyTorch tensor outputs.

```
sentences = [
    "this is the first sentence",
    "instead, this is the second sequence!"
]

# Verified Solution: Tokenize sentences with padding as PyTorch tensors
tokens = tokenizer(sentences, padding=True, return_tensors="pt")
print("Input IDs Tensor:")
print(tokens["input_ids"])
print("Attention Mask Tensor:")
print(tokens["attention_mask"])

# Decode the batch to inspect padding tokens
decoded_batch = tokenizer.batch_decode(tokens["input_ids"])
print("Decoded batch:", decoded_batch)
```

Listing 2.3: Batch tokenization with zero-padding and tensor formatting (Cells 30–36).

The execution output demonstrates how SentencePiece pads the shorter first sentence to match the nine-token length of the second sentence:

```
Input IDs Tensor:
tensor([[  48,   19,    8,  166, 7142,    1,    0,    0,    0],
        [1039,    6,   48,   19,    8,  542, 3817,   55,    1]])

Attention Mask Tensor:
tensor([[1, 1, 1, 1, 1, 1, 0, 0, 0],
        [1, 1, 1, 1, 1, 1, 1, 1, 1]])

Decoded batch: ['this is the first sentence</s><pad><pad><pad>', 'instead, this is
    the second sequence!</s>']
```

Notice that while `tokens["input_ids"]` contains trailing zeros (`<pad>`), `tokens["attention_mask"]` sets exactly those positions to `0`, protecting downstream attention layers from calculating spurious correlations.

2.3 Anatomy of the T5 architecture and embedding geometry

Having parsed discrete text into clean tensor batches, we load the full T5 foundation model: `google-t5/t5-base`.

2.3.1 The T5 unified text-to-text paradigm

Introduced by Raffel et al. [11], T5 reconceptualizes all natural language processing tasks into a unified **Text-to-Text** formulation. Whether performing machine translation, document summarization, grammatical error correction, or sentiment classification, both the input and the target output are represented as text strings:

- **Machine Translation**: `"translate English to German: That is good."` ⟶ `"Das ist gut."`
- **Summarization**: `"summarize: State of the union address..."` ⟶ `"The President discussed..."`
- **Classification**: `"sst2 sentence: The movie was gripping."` ⟶ `"positive"`

Because the task specification is embedded directly within the input prompt as a natural language prefix, a single unified architecture and identical cross-entropy loss function can be applied across all downstream domains without requiring task-specific classification heads.

2.3.2 Model configuration and hyperparameter inspection

Using Hugging Face's `AutoModelForSeq2SeqLM`, we instantiate the model and inspect its architecture:

```python
from transformers import AutoModelForSeq2SeqLM

# Load pretrained sequence-to-sequence model
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
print(type(model))
# Output: <class 'transformers.models.t5.modeling_t5.T5ForConditionalGeneration'>

# Inspect hyperparameter configuration
cfg = model.config
print(f"Hidden size (d_model): {cfg.d_model}")          # 768
print(f"Number of encoder layers: {cfg.num_layers}")    # 12
print(f"Number of decoder layers: {cfg.num_decoder_layers}") # 12
print(f"Number of attention heads: {cfg.num_heads}")    # 12
print(f"Head dimension (d_kv): {cfg.d_kv}")             # 64
print(f"Feed-forward dimension (d_ff): {cfg.d_ff}")     # 3072
print(f"Vocabulary size: {cfg.vocab_size}")             # 32128
```

Listing 2.4: Loading T5-base model and inspecting architectural configuration (Cells 39–44).

> **External deep dive: Dimensional Multi-Head Alignment in T5**

In T5-base, each hidden token representation possesses dimension $d_{\text{model}} = 768$. When entering an attention layer, the token vector is projected into $H = 12$ distinct attention heads.
Each head operates on a projected subspace of dimensionality $d_k = 64$. Notice the exact algebraic equality:

$$H \times d_k = 12 \times 64 = 768 = d_{\text{model}} \tag{2.2}$$

This design guarantees that projecting tokens into 12 distinct representational subspaces and concatenating the multi-head outputs produces a vector of dimension exactly matching d_{model}, allowing residual skip connections ($\mathbf{x} + \text{Sublayer}(\mathbf{x})$) to be added without projection matrices.

2.3.3 Weight sharing across encoder, decoder, and language modeling head

Inspecting the instantiated model reveals a critical memory optimization: **Weight Tying (Weight Sharing)**. In T5, the embedding layer `model.shared` provides the token embedding lookup table for both the encoder and the decoder. Furthermore, the final output linear projection layer (`model.lm_head`), which projects decoder hidden states back to vocabulary logits of dimension 32,128, reuses the exact same weight matrix!

Listing 2.5 displays Cell 49 of `text-01-t5.ipynb`, verifying this memory sharing using Python pointer identities:

```
# Verify memory sharing of embedding layers
is_shared = (id(model.shared) == id(model.encoder.embed_tokens) and
             id(model.shared) == id(model.decoder.embed_tokens))
print("Are encoder and decoder embedding layers physically shared?", is_shared)
# Output: True

# Verify weight sharing with the LM classification head
print("Weight tying check:", id(model.shared.weight) == id(model.lm_head.weight))
# Output: True
```

Listing 2.5: Verifying weight sharing across encoder, decoder, and LM head (Cells 47–49).

Exam insights: Parameter Count Savings via Weight Tying

Why do modern models tie input embedding weights to output projection weights?

1. **Parameter Economy**: An embedding table of size $|\mathcal{V}| \times d_{\text{model}} = 32{,}128 \times 768$ contains approximately 24.67 million parameters. If the encoder embeddings, decoder embeddings, and language modeling head each maintained separate parameters, the model would require $3 \times 24.67 \approx 74$ million parameters solely for token mapping! Weight tying collapses this overhead to a single shared 24.67M matrix, reducing total model parameter footprint by nearly 50 million parameters.

2. **Representational Duality**: Mathematically, if token w_i has embedding vector $\mathbf{e}_i$, predicting next-token probability via the dot product $\mathbf{e}_i^\top \mathbf{h}_{\text{dec}}$ aligns the geometry of the input semantic space with the output prediction space.

2.3.4 Latent embedding space geometry: PCA and cosine similarity

In Cells 51–54 of `text-01-t5.ipynb`, the laboratory explores whether the raw un-contextualized vectors stored in `model.shared` exhibit semantic clustering. We extract embeddings for two semantic clusters: domestic furniture (`chair`, `table`, `plate`, `knife`, `spoon`) and domestic animals (`horse`, `goat`, `sheep`, `cat`, `dog`).

```
words = [
    "chair", "table", "plate", "knife", "spoon",
    "horse", "goat", "sheep", "cat", "dog",
]

# Extract token IDs and lookup embeddings in model.shared
word_tokens = tokenizer(words, return_tensors="pt", padding=True)["input_ids"][:, 0]
token_embeddings = model.shared(word_tokens).cpu().detach().numpy()
print("Embedding matrix shape:", token_embeddings.shape) # (10, 768)

# 1. 2D Principal Component Analysis (PCA) projection
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca = PCA(n_components=2)
X_pca = pca.fit_transform(token_embeddings)

fig, ax = plt.subplots(figsize=(5, 5))
ax.scatter(X_pca[:, 0], X_pca[:, 1])
for i, txt in enumerate(words):
    ax.annotate(txt, (X_pca[i, 0], X_pca[i, 1]))
plt.title("PCA Projection of T5 Word Embeddings")
plt.savefig("pca_embeddings.png")

# 2. Pairwise Cosine Similarity Matrix
from sklearn.metrics.pairwise import cosine_similarity

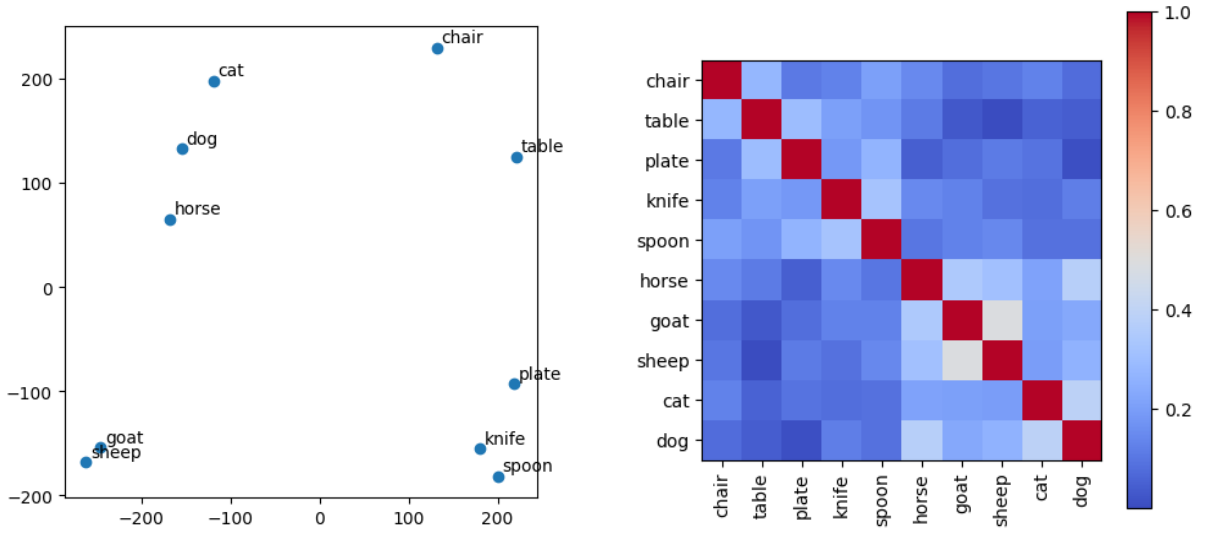
sim_matrix = cosine_similarity(token_embeddings)
fig, ax = plt.subplots(figsize=(5, 5))
cax = ax.imshow(sim_matrix, cmap="coolwarm")
ax.set_xticks(range(len(words)))
ax.set_yticks(range(len(words)))
ax.set_xticklabels(words, rotation=45)
ax.set_yticklabels(words)
fig.colorbar(cax)
plt.title("Cosine Similarity Heatmap")
plt.savefig("cosine_similarity_matrix.png")
```

Listing 2.6: Extracting word embeddings and computing PCA and Cosine Similarity (Cells 51–54).

The resulting visualizations in Figure 2.2 demonstrate strong semantic clustering: animals form a tight cluster with high pairwise cosine similarities (> 0.6), distinctly segregated from household objects.

2.4 Transformer encoder and decoder blocks in T5

The complete T5-base model consists of 12 stacked **Encoder Blocks** and 12 stacked **Decoder Blocks**.



(a) 2D PCA projection of T5 embeddings.

(b) Pairwise Cosine Similarity matrix.

Figure 2.2: Geometric structure of T5-base static embedding representations extracted in Laboratory 02. The latent space clearly segregates household items from mammalian animals, confirming semantic clustering prior to contextual attention processing.

2.4.1 The T5 encoder: Bidirectional self-attention blocks

Each encoder block (`model.encoder.block[i]`) contains two sub-layers:

1. **Bidirectional Self-Attention (`T5LayerSelfAttention`)**: Tokens attend to all other tokens in the input sequence simultaneously. Because the entire input prompt is available upfront, no causal masking is applied. The pre-attention normalization follows a pre-LN design without additive bias parameters (T5 uses RMS-like scaling instead of standard LayerNorm [15]).

2. **Position-wise Feed-Forward Network (`T5LayerFF`)**: A two-layer MLP projecting representations from $d_{\text{model}} = 768 \rightarrow d_{\text{ff}} = 3072$ using a gated ReLU activation, followed by projection back to 768.

2.4.2 The T5 decoder: Causal self-attention and cross-attention

Unlike the encoder, each decoder block (`model.decoder.block[i]`) contains **three** sub-layers:

1. **Causal Self-Attention (`T5LayerSelfAttention`)**: During autoregressive token generation, the model cannot observe future tokens. A lower-triangular causal mask enforces that position i attends only to positions $j \leq i$.

2. **Cross-Attention (`T5LayerCrossAttention`)**: This is the critical connective tissue of the encoder-decoder architecture. The Query vectors ($\mathbf{Q}$) are projected from the decoder's intermediate representations, while the Key ($\mathbf{K}$) and Value ($\mathbf{V}$) vectors are projected from the final output representations of the encoder:

$$\mathbf{Q} = \mathbf{H}_{\text{dec}}\mathbf{W}_q, \quad \mathbf{K} = \mathbf{H}_{\text{enc}}\mathbf{W}_k, \quad \mathbf{V} = \mathbf{H}_{\text{enc}}\mathbf{W}_v \tag{2.3}$$

   This allows every generated output token to dynamically inspect and extract information from the input source prompt.

3. **Position-wise Feed-Forward Network (`T5LayerFF`)**: Identical in dimension to the encoder MLP ($d_{\text{model}} \rightarrow 3072 \rightarrow d_{\text{model}}$).

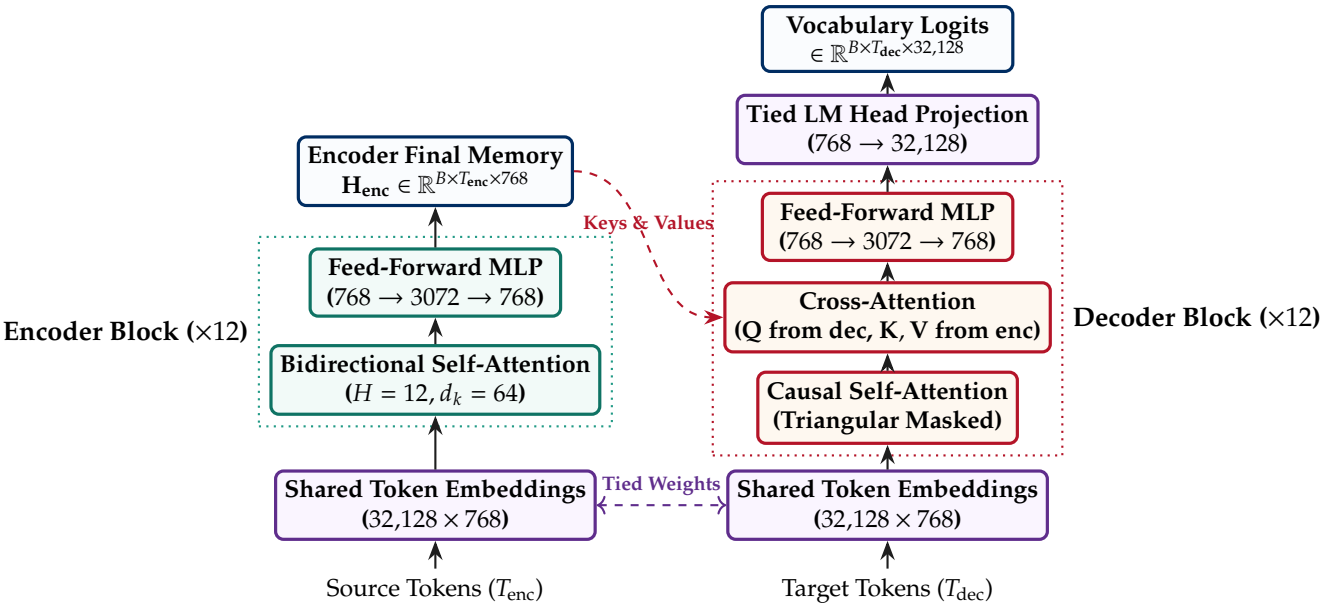


Figure 2.3: The T5 Encoder-Decoder architecture explored in Laboratory 02. The encoder executes bidirectional self-attention across the input prompt; the decoder applies causal self-attention across previously generated tokens and cross-attention over the encoder's final representations; embeddings and classification head are tied.

Figure 2.3 visually synthesizes the structural dataflow connecting input text, token embeddings, bidirectional encoder layers, cross-attention interconnects, causal decoder layers, and the vocabulary logits.

2.5 Autoregressive token generation and decoding dynamics

Foundation models generate text **autoregressively**: token generation proceeds sequentially, where each freshly sampled token is appended to the decoder's context before predicting the subsequent token:

$$P(y_1, y_2, \ldots, y_M \mid x) = \prod_{t=1}^{M} P(y_t \mid y_{<t}, x) \tag{2.4}$$

2.5.1 Logits, vocabularies, and next-token prediction

When passing inputs through the model, the forward call returns a `logits` tensor of shape $[B, T_{\text{dec}}, |\mathcal{V}|]$. Each element $z_{b,t,v}$ represents the unnormalized log-probability of token ID v at step t.

To initialize the generation in T5, the decoder must be primed with an initial start token. In T5, the generation prefix token is the pad token (`decoder_input_ids = [[0]]`).

2.5.2 Step-by-step manual autoregressive generation loop

In Cells 69–85 of `text-01-t5.ipynb`, the laboratory manually executes the autoregressive loop to translate `"translate english to german: hello, how are you?"` into German.

```
import torch

input_sentence = "translate english to german: hello, how are you?"
tokens = tokenizer(input_sentence, return_tensors="pt")
```

```python

# Step 1: Initialize decoder input with pad token
decoder_input_ids = torch.tensor([[ tokenizer.pad_token_id ]])
print("Initial decoder input:", decoder_input_ids) # tensor([[0]])

# Forward pass through encoder and first decoder step
output = model(**tokens, decoder_input_ids=decoder_input_ids)
print("Logits shape:", output.logits.shape) # torch.Size([1, 1, 32128])

# Extract predicted token at position 0 via greedy argmax
max_proba_token = output.logits[0, 0].argmax()
print(f"Token ID: {max_proba_token.item()} -> Word:
    '{tokenizer.decode(max_proba_token)}'")
# Output: Token ID: 21820 -> Word: 'Hallo'

# Step 2: Append newly generated token to decoder context
decoder_input_ids = torch.hstack([decoder_input_ids, max_proba_token.view(1, 1)])
print("Updated decoder inputs:", decoder_input_ids) # tensor([[0, 21820]])

# Step 3: Run full manual loop until </s> or max_length
decoder_input_ids = torch.tensor([[ tokenizer.pad_token_id ]])
max_length = 50
i = 0

while i < max_length and decoder_input_ids[0, -1] != tokenizer.eos_token_id:
    output = model(**tokens, decoder_input_ids=decoder_input_ids)
    next_token = output.logits[0, -1].argmax().view(1, 1)
    decoder_input_ids = torch.hstack([decoder_input_ids, next_token])
    i += 1

translated_text = tokenizer.decode(decoder_input_ids[0], skip_special_tokens=True)
print("Manually generated translation:", translated_text)
# Output: "Hallo, wie geht es Ihnen?"
```

Listing 2.7: Step-by-step manual autoregressive decoding implementation (Cells 69–85).

2.5.3 The `model.generate()` API and Key-Value (KV) caching

While the manual `while` loop in Listing 2.7 illustrates the mathematical logic of autoregression, it is computationally naive. Notice what occurs at iteration t:

1. The model re-encodes the input sequence (or passes the full encoder states).
2. More critically, the decoder recomputes self-attention projections ($\mathbf{Q}, \mathbf{K}, \mathbf{V}$) for all previous $t-1$ tokens, even though their causal past has not changed!

Consequently, generating a sequence of length T_{dec} naively requires $O(T_{\text{dec}}^2)$ attention operations.

Hugging Face's optimized `model.generate()` method circumvents this recomputation by activating **Key-Value Caching (KV-Cache)** via the internal argument `use_cache=True`:

```python
# Production generation with KV-cache and beam search / greedy decoding
out_tokens = model.generate(**tokens)
translation = tokenizer.batch_decode(out_tokens, skip_special_tokens=True)
print("Optimized generation:", translation[0])
# Output: "Hallo, wie geht es Ihnen?"
```

Listing 2.8: High-level optimized generation using `model.generate()` (Cells 87–88).

Exam insights: KV-Cache Mechanics and Complexity Comparison

On examinations, students are frequently asked to contrast naive autoregressive generation with KV-cached decoding:

- **Naive Generation (Without Cache)**: At step t, the decoder processes a sequence of length t. The self-attention matrix requires computing $t \times t$ dot products. Over an entire generation of M tokens, total attention operations scale as:

$$\sum_{t=1}^{M} O(t) = O(M^2) \tag{2.5}$$

- **Cached Generation (With KV-Cache)**: At step t, the decoder passes only the single latest token ($T = 1$). Its query $\mathbf{q}_t$ attends to previously stored key and value vectors ($\mathbf{K}_{<t}, \mathbf{V}_{<t}$), and its newly computed $\mathbf{k}_t, \mathbf{v}_t$ are appended to the cache. Each step takes $O(t)$ operations for dot products, but avoids re-running linear projections for the previous $t-1$ tokens, drastically slashing wall-clock latency.

2.6 Cross-attention weight extraction and interpretability

One of the most remarkable properties of attention-based architectures is their inherent **mechanistic interpretability**: by inspecting attention weight matrices, we can observe precisely which source tokens the model focuses upon when synthesizing each output token.

2.6.1 Accessing internal cross-attention tensors

To extract attention tensors in Hugging Face, we instantiate the model with `output_attentions=True`:

```
# Load model with attention extraction enabled
model = AutoModelForSeq2SeqLM.from_pretrained(model_name, output_attentions=True)

# Run forward pass for the first generation step
output = model(**tokens, decoder_input_ids=torch.tensor([[ tokenizer.pad_token_id ]]))

# output.cross_attentions is a tuple of length 12 (one tensor per decoder layer)
print("Number of cross-attention layers:", len(output.cross_attentions)) # 12

# Inspect tensor dimensions of Layer 0
layer0_attn = output.cross_attentions[0]
print("Cross-attention shape:", layer0_attn.shape)
# Output: torch.Size([1, 12, 1, 13])
```

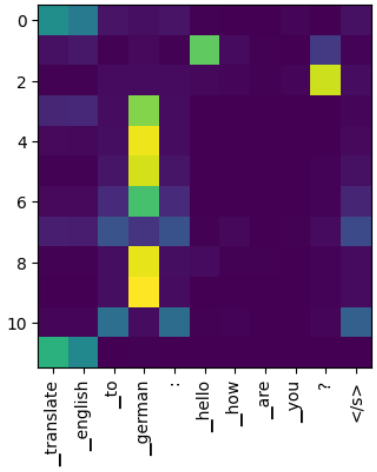
Listing 2.9: Extracting cross-attention weights during forward pass (Cells 90–96).

The 4D shape $[B, H, T_{\text{dec}}, T_{\text{enc}}] = [1, 12, 1, 13]$ reflects:

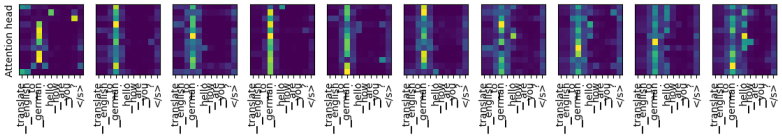
- $B = 1$: Batch size.
- $H = 12$: Number of distinct attention heads in the decoder layer.
- $T_{\text{dec}} = 1$: Single query token in the decoder (the initial pad token generating `"Hallo"`).
- $T_{\text{enc}} = 13$: Thirteen tokens in the input prompt (`translate`, `english`, `to`, `german`, `:`, `hello`, `,`, `how`, `are`, `you`, `?`, `</s>`).

2.6.2 Early vs late layer attention patterns

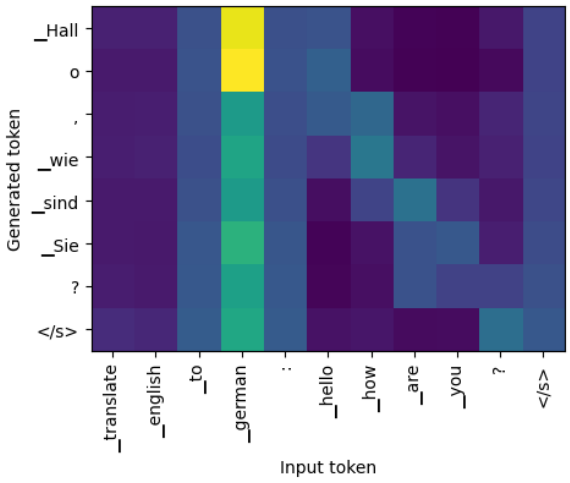
Cells 98–104 of `text-01-t5.ipynb` plot the attention distributions across different layers and across all generated output tokens, shown in Figure 2.4.



(a) Layer 0 cross-attention.



(b) Cross-attention heatmaps across layers 0 to 9.



(c) Average cross-attention for full translation.

Figure 2.4: Cross-attention visualization in T5. Early layers attend broadly to task prefix instructions (`translate`, `english`, `german`), whereas deeper layers (6–11) dynamically align target tokens (`Hallo`) with their source counterparts (`hello`).

The empirical findings from Figure 2.4 reveal a distinct functional hierarchy across Transformer depth:

1. **Shallow Layers (Layers 0–3)**: Attention heads focus heavily on the **task prefix prompt tokens** (`"translate"`, `"english"`, `"to"`, `"german"`). The model is determining the functional modality of the task (i.e., routing computational state toward German translation rather than English summarization).

2. **Deep Layers (Layers 6–11)**: Attention heads dynamically shift focus to the **content tokens**. When generating `"Hallo"`, maximum attention weight concentrates sharply on `"hello"`. When generating `"wie"`, attention shifts to `"how"`.

2.7 Scaled dot-product attention from scratch: Mathematical foundations

Having explored Hugging Face's production implementation of T5, we transition to Module 2 of the laboratory (`text-02-attention.ipynb`): isolating and constructing the **Scaled Dot-Product Attention mechanism** from first principles in pure PyTorch.

2.7.1 Query, Key, and Value vectors: Database retrieval analogy

The attention mechanism models sequence interactions using the conceptual framework of a soft, differentiable **database query-retrieval system** [2]:

Intuition & analogy: The Query, Key, Value Information Retrieval Analogy

Consider searching for a video on YouTube:

- **Query (q)**: The text string you type into the search bar. It represents what the current token is *looking for*.
- **Key (k)**: The video title and metadata tags stored on YouTube's servers. It represents the *indexing handle* describing what each candidate token contains.
- **Value (v)**: The actual video content streamed back to your screen. It represents the *substantive information* propagated forward if a match occurs.

In self-attention, we evaluate the compatibility between Query $\mathbf{q}_i$ and Key $\mathbf{k}_j$ via their dot product $\mathbf{q}_i^\top \mathbf{k}_j$. Passing these compatibilities through a softmax yields normalized weights α_{ij}. The output representation is the weighted mixture of Value vectors: $\mathbf{o}_i = \sum_j \alpha_{ij} \mathbf{v}_j$.

2.7.2 Mathematical derivation and the scaling factor $\frac{1}{\sqrt{d_k}}$

Given query matrix $\mathbf{Q} \in \mathbb{R}^{T \times d_k}$, key matrix $\mathbf{K} \in \mathbb{R}^{T \times d_k}$, and value matrix $\mathbf{V} \in \mathbb{R}^{T \times d_v}$, the scaled dot-product attention equation is defined as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V} \tag{2.6}$$

Theorem 2.1: *Variance Preservation of the Scaling Factor*

Let components of query vector $\mathbf{q} = [q_1, \ldots, q_{d_k}]$ and key vector $\mathbf{k} = [k_1, \ldots, k_{d_k}]$ be independent, identically distributed random variables with zero mean and unit variance:

$$\mathbb{E}[q_m] = \mathbb{E}[k_m] = 0, \quad \mathbb{V}\text{ar}(q_m) = \mathbb{V}\text{ar}(k_m) = 1, \quad \forall m \in \{1, \ldots, d_k\} \tag{2.7}$$

Then the dot product $S = \mathbf{q}^\top \mathbf{k} = \sum_{m=1}^{d_k} q_m k_m$ has expected value:

$$\mathbb{E}[S] = \sum_{m=1}^{d_k} \mathbb{E}[q_m k_m] = \sum_{m=1}^{d_k} \mathbb{E}[q_m]\,\mathbb{E}[k_m] = 0 \tag{2.8}$$

and variance:

$$\begin{aligned}\mathbb{V}\mathrm{ar}(S) &= \sum_{m=1}^{d_k} \mathbb{V}\mathrm{ar}(q_m k_m) \\ &= \sum_{m=1}^{d_k} \left(\mathbb{V}\mathrm{ar}(q_m)\,\mathbb{V}\mathrm{ar}(k_m) + \mathbb{V}\mathrm{ar}(q_m)\,\mathbb{E}[k_m]^2 + \mathbb{V}\mathrm{ar}(k_m)\,\mathbb{E}[q_m]^2\right) \\ &= \sum_{m=1}^{d_k} (1 \cdot 1 + 0 + 0) = \sum_{m=1}^{d_k} 1 = d_k \end{aligned} \tag{2.9}$$

Consequently, scaling the dot product by $\frac{1}{\sqrt{d_k}}$ yields a stabilized score $Z = \frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}}$ with strictly unit variance:

$$\mathbb{V}\mathrm{ar}(Z) = \mathbb{V}\mathrm{ar}\left(\frac{S}{\sqrt{d_k}}\right) = \frac{1}{d_k}\,\mathbb{V}\mathrm{ar}(S) = \frac{d_k}{d_k} = 1 \tag{2.10}$$

Exam insights: Softmax Saturation and Vanishing Gradients

Why is dividing by $\sqrt{d_k}$ mandatory when d_k is large (e.g., $d_k = 64$ or 128)?
If dot products are not scaled, for large d_k the magnitude of scores $|S|$ grows proportionally to $\sqrt{d_k}$. When large values (e.g., +30 or −30) enter the softmax function:

$$\mathrm{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \tag{2.11}$$

the exponential function causes the probability distribution to collapse into a one-hot distribution (the maximum value receives probability ≈ 1, all others ≈ 0).
In this saturated region, the Jacobian derivative of the softmax function approaches zero:

$$\frac{\partial\,\mathrm{softmax}(\mathbf{z})_i}{\partial z_j} = \mathrm{softmax}(\mathbf{z})_i(\delta_{ij} - \mathrm{softmax}(\mathbf{z})_j) \approx 0 \tag{2.12}$$

This induces severe **vanishing gradients**, preventing the optimizer from updating upstream query and key projection matrices during backpropagation! Dividing by $\sqrt{d_k}$ prevents saturation and stabilizes learning dynamics.

2.7.3 TikZ computation graph of Scaled Dot-Product Attention

Figure 2.5 illustrates the explicit dataflow, showing how the intermediate score matrix $\mathbf{QK}^\top$ forms an all-pairs quadratic matrix of shape $[B, T, T]$ before being multiplied by $\mathbf{V}$.

2.8 Synthetic time series modeling with periodic jumps

To rigorously test the attention mechanism's ability to model sequential relationships, Module 2 formulates a challenging synthetic regression task: predicting the next value in a sequence exhibiting periodic, discontinuous step jumps.

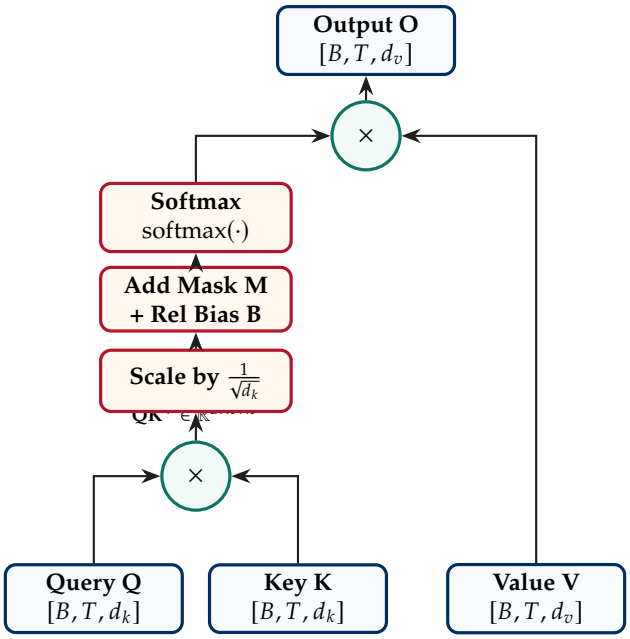


Figure 2.5: Computational graph of the Scaled Dot-Product Attention mechanism with explicit tensor dimensions.

2.8.1 Problem formulation: Discontinuous periodic sequence prediction

Standard autoregressive patterns (such as pure sinusoidal oscillations or linear trends) can often be solved trivially by linear autoregressive filters. To force the model to rely on non-local sequential memory, the dataset generates signals with abrupt jumps:

- Sequence length: $T = 40$.
- A baseline signal initializes at zero.
- An initial jump of magnitude $5 + \epsilon$ occurs at a random offset $t_0 \in [0, 3)$.
- Subsequent jumps occur strictly every val $= 3$ timesteps ($t_0 + 3, t_0 + 6, t_0 + 9, \ldots$), alternating between 0 and $5 + \epsilon$.

The target $y \in \mathbb{R}$ is the signal value at timestep $T = 40$. To accurately predict y, the model cannot rely solely on the most recent step; it must determine whether timestep 40 corresponds to a jump step relative to the sequence's periodic phase.

2.8.2 Verified code: Custom `TimeSeriesDataset` and `DataLoader`

Listing 2.10 displays the verified implementation from Cell 7 of `text-02-attention.ipynb`.

```
import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

class TimeSeriesDataset(torch.utils.data.Dataset):
    def __init__(self, seq_len, num_samples):
        val = 3          # Periodicity of jumps
        initial_jump = 5 # Jump amplitude

        self.seq_len = seq_len
```

```
        self.data = []
        for _ in range(num_samples):
            time_series = np.zeros(seq_len + 1)

            # Introduce the first jump at a random point between 0 and val
            first_jump = np.random.randint(0, val)
            time_series[first_jump] += initial_jump + np.random.rand()

            # Continue introducing jumps every val timesteps after first jump
            for i in range(first_jump + val, seq_len + 1, val):
                jump = 0 if i % 2 == 1 else initial_jump
                time_series[i] += jump + np.random.rand()

            self.data.append(time_series)

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        series = self.data[idx]
        # Return input sequence (first seq_len values) and target scalar (final value)
        return (torch.tensor(series[:self.seq_len], dtype=torch.float32),
                torch.tensor(series[self.seq_len], dtype=torch.float32))

# Visualize a single generated sequence
dataset = TimeSeriesDataset(seq_len=40, num_samples=1)
x, y = dataset[0]
plt.figure(figsize=(7, 3))
plt.plot(range(40), x, label='x (input sequence)')
plt.scatter([40], [y], color='red', label=f'y (target: {y:.2f})')
plt.title(f"Synthetic Time Series with Periodic Jumps (Period=3)")
plt.legend()
plt.savefig("synthetic_time_series_sample.png")
```

Listing 2.10: Verified implementation of `TimeSeriesDataset` with periodic jumps (Cells 7–8).

2.8.3 Implementing custom `ScaledDotProductAttention` and `AttentionHead`

Next, we construct the attention layers from scratch. Listing 2.11 presents the verified implementations from Cells 14 and 18:

```
class ScaledDotProductAttention(nn.Module):
    def __init__(self, seq_len, d_k, other_positional_encodings_present):
        super(ScaledDotProductAttention, self).__init__()
        self.d_k = d_k
        self.seq_len = seq_len
        self.other_positional_encodings_present = other_positional_encodings_present

        # Learnable relative positional encoding matrix [1, seq_len, seq_len]
        self.rel_layer = nn.Parameter(torch.randn(1, self.seq_len, self.seq_len))

    def forward(self, query, key, value):
        # 1. Compute dot products scaled by sqrt(d_k)
        scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(self.d_k)

        # 2. Add relative positional bias if absolute encodings are not present
        if not self.other_positional_encodings_present:
            scores = scores + self.rel_layer
```

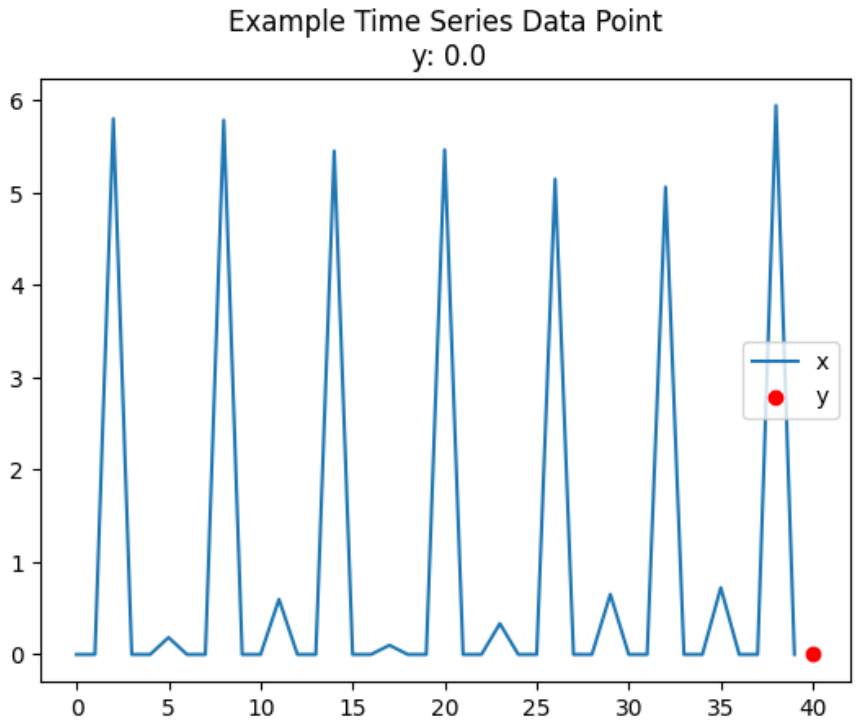


Figure 2.6: Sample time series from the synthetic dataset generated in Laboratory 02. The sequence exhibits periodic spikes every 3 steps with randomized starting phase, requiring the attention mechanism to learn shift-invariant relative offsets.

```
        # 3. Softmax normalization across sequence dimension
        attention_weights = torch.softmax(scores, dim=-1)

        # 4. Multiply by Value matrix
        output = torch.matmul(attention_weights, value)
        return output, attention_weights


class AttentionHead(nn.Module):
    def __init__(self, d_model, seq_len, other_positional_encodings_present):
        super(AttentionHead, self).__init__()
        self.d_k = d_model
        self.seq_len = seq_len

        # Linear projections for Query, Key, and Value
        self.query_layer = nn.Linear(d_model, d_model)
        self.key_layer = nn.Linear(d_model, d_model)
        self.value_layer = nn.Linear(d_model, d_model)

        # Scaled dot-product core
        self.scaled_dot_attention = ScaledDotProductAttention(
            self.seq_len, self.d_k, other_positional_encodings_present
        )

        # Final projection layer
        self.fc_out = nn.Linear(d_model, d_model)

    def forward(self, query, key, value):
```

```
        Q = self.query_layer(query)
        K = self.key_layer(key)
        V = self.value_layer(value)

        attn_out, attn_weights = self.scaled_dot_attention(Q, K, V)
        output = self.fc_out(attn_out)
        return output, attn_weights
```

Listing 2.11: Implementing `ScaledDotProductAttention` and `AttentionHead` (Cells 14–18).

2.8.4 Transformer Encoder Block and Time Series Architecture

We assemble the attention head into a full `TransformerEncoderBlock` with residual connections, LayerNorm, and a feed-forward MLP (Listing 2.12), followed by the top-level `Transformer` regression model:

```
class TransformerEncoderBlock(nn.Module):
    def __init__(self, d_model, dim_feedforward, seq_len,
        other_positional_encodings_present, dropout=0.1):
        super(TransformerEncoderBlock, self).__init__()
        self.attention = AttentionHead(d_model, seq_len,
            other_positional_encodings_present)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)

        # Feed-forward network
        self.ffn = nn.Sequential(
            nn.Linear(d_model, dim_feedforward),
            nn.ReLU(),
            nn.Linear(dim_feedforward, d_model)
        )
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # Multi-head attention with residual skip and LayerNorm
        attn_out, attn_weights = self.attention(x, x, x)
        x = self.norm1(x + self.dropout(attn_out))

        # Position-wise feed-forward with residual skip and LayerNorm
        ffn_out = self.ffn(x)
        output = self.norm2(x + self.dropout(ffn_out))
        return output, attn_weights


class Transformer(nn.Module):
    def __init__(self, input_size, d_model, dim_feedforward, num_layers, seq_len,
                 positional_encoding=None, dropout=0.1, max_len=5000):
        super(Transformer, self).__init__()
        self.positional_encoding = positional_encoding

        # Linear projection: scalar time series feature -> d_model
        self.embedding = nn.Linear(input_size, d_model)

        if self.positional_encoding is not None:
            self.pos_encoder = self.positional_encoding(d_model, max_len)

        # Stacked Transformer Encoder blocks
        self.encoder_layers = nn.ModuleList([
```

```
            TransformerEncoderBlock(
                d_model, dim_feedforward, seq_len,
                False if positional_encoding is None else True,
                dropout
            ) for _ in range(num_layers)
        ])

        # Linear head projecting last timestep representation to predicted scalar
        self.fc_out = nn.Linear(d_model, 1)

    def forward(self, x):
        # x shape: [batch_size, seq_len, input_size]
        x = self.embedding(x)
        if self.positional_encoding is not None:
            x = self.pos_encoder(x)

        all_attention_weights = []
        for layer in self.encoder_layers:
            x, attn_weights = layer(x)
            all_attention_weights.append(attn_weights)

        # Predict from final timestep token (seq_len - 1)
        output = self.fc_out(x[:, -1])
        return output, all_attention_weights
```

Listing 2.12: Complete `TransformerEncoderBlock` and `Transformer` model (Cells 21–24).

2.9 Relative vs absolute positional encodings: Theory and empirical proof

A central conceptual breakthrough of Laboratory 02 is understanding **Permutation Equivariance** and why the choice of positional representation dramatically dictates sequence learning capability.

2.9.1 The permutation equivariance dilemma

Standard self-attention is strictly **permutation equivariant**: if the rows of input matrix $\mathbf{X}$ are permuted by permutation matrix $\mathbf{P}$, the output is identically permuted:

$$\text{Attention}(\mathbf{PQ}, \mathbf{PK}, \mathbf{PV}) = \mathbf{P}\,\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \tag{2.13}$$

In other words, without positional information, self-attention treats an input sequence as an unordered set (a bag of tokens)! To model sequential ordering, position must be injected into the architecture.

2.9.2 Absolute sinusoidal positional encodings (Vaswani et al. 2017)

In the original Transformer [2], Vaswani et al. added deterministic sinusoidal vectors to input embeddings:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{2.14}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{2.15}$$

Listing 2.13 displays the implementation from Cell 36:

```python
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super(PositionalEncoding, self).__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() *
            (-math.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        pe = pe.unsqueeze(0).transpose(0, 1) # [max_len, 1, d_model]
        self.register_buffer('pe', pe)

    def forward(self, x):
        # Add absolute positional vector directly to embedded inputs
        return x + self.pe[:x.size(0), :].repeat(1, x.shape[1], 1)
```

Listing 2.13: Absolute sinusoidal positional encoding implementation (Cell 36).

2.9.3 Relative positional encodings (Shaw et al. 2018 / T5)

Rather than adding absolute coordinates to inputs, **Relative Positional Encodings** [11, 12] recognize that what matters in language and sequence modeling is not whether a word is at index 37, but rather that word j is **3 positions before** word i ($i - j = 3$).

In our implemented relative attention layer, an offset bias matrix $\mathbf{B} \in \mathbb{R}^{T \times T}$ is added directly to the attention logits:

$$\text{Score}_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}} + B_{i,j} \tag{2.16}$$

where $B_{i,j}$ directly models the pairwise relative distance between timesteps i and j.

2.9.4 Empirical convergence benchmark: Relative (0.036) vs Absolute (5.417)

Both models were trained on identical synthetic datasets ($N = 1000$ sequences, sequence length $T = 40$, batch size 128, Adam optimizer, learning rate 5×10^{-4}, MSE loss) across 15 epochs.

The empirical loss trajectory recorded during Laboratory 02 is presented in Table 2.2 and plotted via PGFPlots in Figure 2.7.

2.9.5 Why relative attention dominates: Invariance under phase shifts

The dramatic divergence in Figure 2.7 demonstrates a core principle of sequence modeling:

- **Phase Invariance**: In our dataset, the initial jump occurs at a random offset $t_0 \in \{0, 1, 2\}$. Therefore, in Sample A, jumps might occur at indices $0, 3, 6, 9, \ldots, 39$. In Sample B, jumps might occur at indices $2, 5, 8, 11, \ldots, 38$.
- **Failure of Absolute Encodings**: The absolute model associates each step with fixed index coordinates (PE_{pos}). Because a jump occurs at index 39 in Sample A but at index 38 in Sample B, absolute index coordinates provide contradictory signals across samples. The model cannot generalize, causing training loss to plateau at 5.4174.
- **Success of Relative Encodings**: The relative model evaluates pairwise distances $i - j$. In every single sample, regardless of where the sequence started, a jump at step i is strictly correlated with what occurred at step $i - 3$! The relative bias matrix $B_{i,j}$ easily learns high affinity for relative offsets $\Delta = -3$ and $\Delta = -6$, allowing the model to achieve 0.0363 loss.

Epoch	Relative Positional Encoding Loss	Absolute Sinusoidal Encoding Loss
1	7.3971	7.1401
2	6.1423	6.5387
3	5.3159	6.2578
4	4.4852	5.9696
5	3.7367	5.8090
6	2.3744	5.7272
7	1.0627	5.7824
8	0.3977	5.6585
9	0.2739	5.5258
10	0.1367	5.6687
11	0.0736	5.5961
12	0.0533	5.4340
13	0.0436	5.3413
14	0.0380	5.3534
15	**0.0363**	**5.4174**

Table 2.2: Empirical loss trajectory over 15 training epochs in Laboratory 02. The Relative Positional Encoding model achieves near-zero error (0.0363), while Absolute Sinusoidal Encodings completely fail to converge (5.4174).

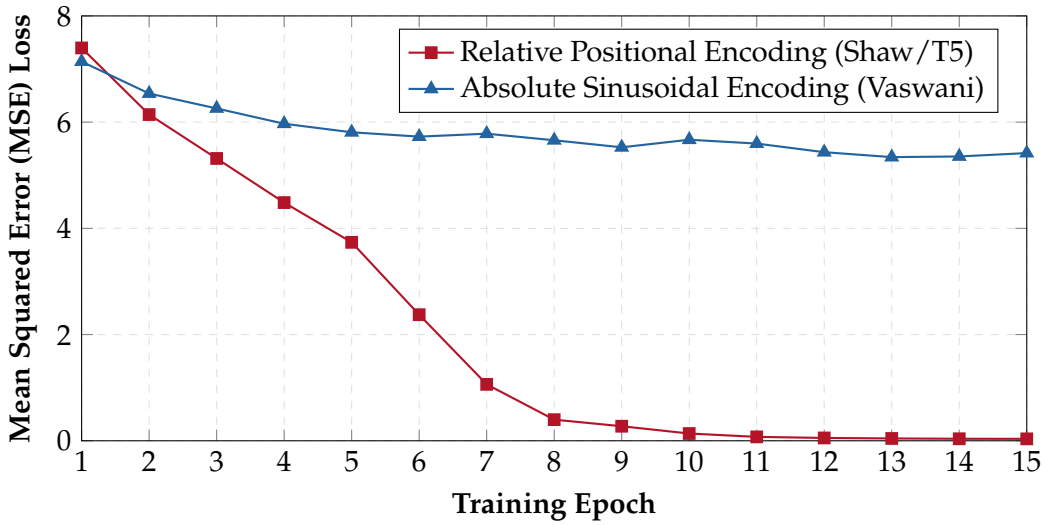
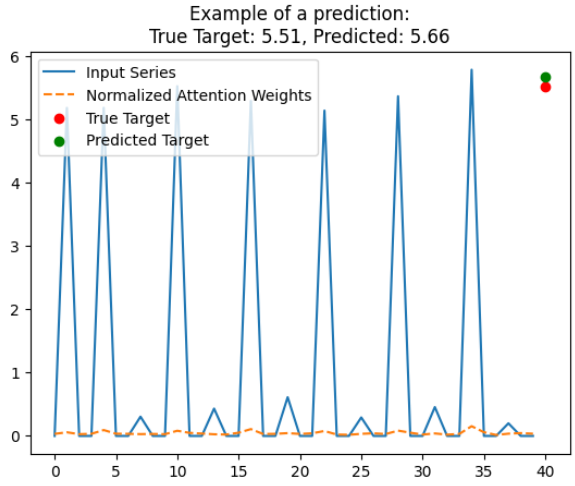
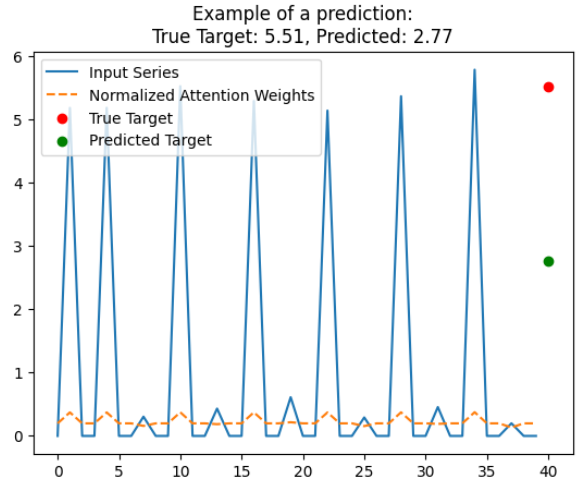


Figure 2.7: Analytical PGFPlots convergence benchmark from Laboratory 02. The model equipped with learnable relative bias converges smoothly from 7.3971 $\rightarrow$ 0.0363, successfully locking onto the period-3 invariant. The absolute sinusoidal model stagnates at 5.4174 because randomized phase shifts destroy absolute index correlations.



(a) Attention weights: Relative Positional Encoding.



(b) Attention weights: Absolute Positional Encoding.

Figure 2.8: Normalized attention weight profiles over input sequences. The relative model (a) concentrates sharp attention spikes exactly on periodic jump predecessors ($t-3, t-6$), whereas the absolute model (b) produces diffuse, uninformative attention spread.

2.9.6 Modern extensions: RoPE and ALiBi

While Shaw et al. [12] and T5 introduced relative position bias matrices, modern open-source foundation models deploy more advanced formulations:

1. **Rotary Position Embeddings (RoPE)** [16]: Deployed in LLaMA, Mistral, and Gemma. Instead of adding a bias to attention logits, RoPE rotates Query and Key vectors in the 2D complex plane by an angle proportional to their sequence position:

$$\mathbf{q}_m = \mathbf{R}^d_{\Theta,m}\mathbf{W}_q\mathbf{x}_m, \quad \mathbf{k}_n = \mathbf{R}^d_{\Theta,n}\mathbf{W}_k\mathbf{x}_n \tag{2.17}$$

   The inner product $\mathbf{q}_m^\top\mathbf{k}_n$ naturally depends strictly on the relative distance $m-n$, combining the simplicity of absolute input vectors with the theoretical advantages of relative position representations.

2. **ALiBi (Attention with Linear Biases)** [17]: Deployed in MPT and BLOOM. ALiBi adds a non-learned static negative penalty proportional to relative distance directly to attention logits:

$$\text{Score}_{ij} = \frac{\mathbf{q}_i^\top\mathbf{k}_j}{\sqrt{d_k}} - m\cdot|i-j| \tag{2.18}$$

   where m is a head-specific slope. This achieves state-of-the-art sequence length extrapolation without requiring any positional parameter training.

2.10 Instructor insights, exam takeaways, and engineering checklist

2.10.1 Key takeaways from the teaching assistant lecture walkthrough

During the laboratory live session, teaching assistant Claudio Savelli emphasized several critical software engineering takeaways:

1. **SentencePiece Detokenization**: The underscore prefix _ preserves whitespace boundary information. Decoding never guesses where spaces belonged; detokenization is lossless.

2. **VRAM Saturation via Batch Outliers**: A single long sentence in a batch forces all other sequences to be zero-padded to that length, wasting substantial memory and GPU FLOPs. Always inspect sequence length histograms before batching.
3. **Dimensional Matching in Multi-Head Layers**: Ensure that d_{model} is cleanly divisible by H. In T5, $768 = 12 \times 64$.
4. **Weight Sharing Verification**: T5 saves tens of millions of parameters by sharing a single tensor between encoder embeddings, decoder embeddings, and the language modeling classification head.

2.10.2 Recurrent exam questions and traps

Exam insights: Common Exam Traps on Attention and Seq2Seq Models

- **Trap 1: Cross-Attention Dimensions**: On exams, students often incorrectly assume cross-attention has shape $[B, H, T_{\text{enc}}, T_{\text{enc}}]$. Remember: in cross-attention, Queries originate from the **decoder** (T_{dec}), while Keys and Values originate from the **encoder** (T_{enc}). The resulting attention matrix has shape:

$$[B, H, T_{\text{dec}}, T_{\text{enc}}] \tag{2.19}$$

- **Trap 2: Forgetting the Causal Mask in Autoregression**: If a student implements a custom decoder without applying lower-triangular masking ($M_{ij} = -\infty$ for $j > i$), the decoder cheats during training by looking ahead at ground truth future tokens, resulting in perfect training loss but completely corrupted inference generation.
- **Trap 3: Why Divide by $\sqrt{d_k}$**: Be prepared to write down Theorem 2.1. You must demonstrate that without scaling, the dot-product variance scales with d_k, causing softmax saturation and vanishing backpropagation gradients.
- **Trap 4: Relative vs Absolute Positional Encodings**: Absolute encodings assign fixed coordinates to sequence indices; relative encodings model pairwise distance offsets $i - j$. In periodic or translation-invariant sequences with varying start offsets, absolute encodings fail while relative encodings succeed.

2.10.3 Engineering checklist: Validating Transformer implementations

When building or fine-tuning Transformer models in PyTorch, verify each step against this engineering checklist:

- ☐ **Padding Mask Integration**: Have you verified that pad tokens (`id=0`) have attention mask set to `0` in both encoder and decoder?
- ☐ **Scaling Factor Check**: Is your dot-product division strictly $\frac{1}{\sqrt{d_k}}$ and not $\frac{1}{\sqrt{d_{\text{model}}}}$?
- ☐ **KV-Cache Deployment**: For production inference loops, is `use_cache=True` activated to avoid $O(T^2)$ recomputation?
- ☐ **Weight Tying Registration**: If tying embedding weights to classification heads, ensure they share identical memory addresses (`lm_head.weight = shared.weight`).
- ☐ **Permutation Invariance Test**: If building custom attention architectures, pass a permuted sequence to verify whether positional encodings correctly break permutation symmetry.

2.10.4 Bridge to Laboratory 03

In Laboratory 03, we extend these foundational Transformer mechanics to full-scale modern language modeling architectures: delving deep into the bidirectional masked language modeling of BERT [6], the causal autoregressive pretraining of GPT-2 [8], and parameter-efficient fine-tuning on downstream software engineering tasks.

Part II

Large language models for software engineering

Bibliography

[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.

[3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997.

[4] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning (ICML)*, pages 1310–1318, 2013.

[5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 16344–16359, 2022.

[6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186, 2019.

[7] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, 2020.

[8] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[9] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[10] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[11] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL https://jmlr.org/papers/v21/19-460.html.

[12] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 464–468, 2018. URL https://arxiv.org/abs/1803.02155.

[13] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.

[14] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018.

[15] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. URL https://arxiv.org/abs/1607.06450.

[16] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. URL https://arxiv.org/abs/2104.09864.

[17] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2108.12409.